

LN-GAP

Lightning governed by arbitrary programs

waxwing

September 2026

Version 1.0 (draft)

Contents

1	Abstract	2
2	Introduction	3
2.1	The problem of 2-party games on Lightning	3
2.1.1	ForceMove is not the full solution	4
2.1.2	Can we do ForceMove offchain?	5
2.2	From a proof of publication ledger to a venue	6
2.2.1	Proof of Work	7
2.2.2	Bonded validator quorum	8
2.3	What the venue attests to	9
3	The venue	11
3.1	What: attestations keyed by contract and move	11
3.2	When: due times and flags	12
3.3	A better-conditioned oracle problem	13
3.4	The registry	14
3.5	Incentives	14
4	Contract outputs and the dispute graph	15
4.1	Keys and venue data fixed at open	16
4.2	The commitment transaction	16
4.3	The claim output and the refutation	17
4.4	The refuted output	19
4.5	Pre-signing	19
4.6	How the points and signatures connect	20
4.7	The fee lock output	22
4.8	Measured sizes	22
5	A game from open to close	23
6	Beyond games: a public clock and a seal	25

6.1	Two different services	25
6.2	Shared facts, and what defends them	25
7	Storage contracts: Filecoin without an altcoin	26
8	Hidden information: blackjack	27
9	A sealed-bid auction	29
10	Arbitrary programs: disputing a zero-knowledge verifier	30
10.1	Claims that carry their own proof	32
10.2	Channels instead of bridges	32
11	Security	33
12	Weakening the majority assumption	37
12.1	The threshold adversary	37
12.2	Committees drawn from a roster	38
12.3	The threshold: why a bare majority	39
12.4	Why not publish on Bitcoin?	40
12.5	A third layer	41
12.6	What remains	42
13	Related work	42
14	Conclusion	43
A	The chess disprove family, in full	45
A.1	What a disprove leaf sees	45
A.2	The predicates	46
A.3	A worked disproof: a bishop through a pawn	48
A.4	The same disproof, as bytes	49

1. Abstract

We propose, without any consensus change to Bitcoin, offchain bilateral contracts in Bitcoin that depend on arbitrary computation, calling this general paradigm LN-GAP (Lightning Network governed by arbitrary programs). That general idea is not new but we describe here a distinct mechanism to achieve $O(1)$ -transactions to resolve disputed contracts onchain (that is, there is no onchain bisection), based on a *venue*: a committee of validators that acts as a public clock for each contract, not as a ledger. The onchain dispute resolution depends on a threshold attestation of the validators to absence of a published entry, and a 1 of n positive signature over the entry in a novel way (we call this an elliptic curve one time signature, or EC-OTS), and in so doing allows the application of the ForceMove paradigm for the offchain game with an independent timeout clock that does not rely on facts about the Bitcoin chain itself (thus not requiring Bitcoin SPV proofs). The advantages of this construction are: publishing entries requires only 1 of n liveness, contracts are fully denominated in bitcoin and trust-minimised: they rely on the honesty of a majority of the venue’s validators (a committee that can be drawn

at random, per contract, from a large bonded roster), whose misbehaviour is attributable and who themselves are not required to know anything about the bilateral contract details, and fraudulent equivocation is provable. Those validators are motivated by both positive fees (paid, in bitcoin, atomically with publication in LN channels) and negatively by bonds (timelocked funds, whose owners lose their future fees when fraud is attributed, and which can optionally be made burnable).

2. Introduction

Start from the **known good** - that Lightning as a payments technology works, and works well. It aids privacy and scalability, if not perfectly addressing both, it addresses them to a large extent, at the cost of extra infrastructure for users and service providers. Once that infrastructure is in place, the *payments* UI/workflow for users is as smooth as it should be¹.

What's missing? Lightning already provides private, fast and cheap payments. In vague terms, I would argue that all it really misses is “programmable extensibility”. Take as an example this goal:

The customer wants to pay a storage service provider once per day, in sats, but wants the service provider to prove that they are upholding their agreement (with a response to a challenge for an inclusion proof), and if they fail to do so after a time window, their deposit can be taken.

This is basically “Filecoin without an altcoin”, as designed in this paper (Section 7).

2.1. The problem of 2-party games on Lightning

Suppose you want to play a game on Lightning, like chess, for a monetary reward, but don't want to have to trust anyone; the money should move specifically according to the agreed game rules.

I mention chess not because it's a good game to bet on (it isn't!) but because it's a game that's sufficiently complex that expressing it in onchain transactions (most specifically, proving that a move is illegal; proving that a move is legal would be way worse!) gets very expensive and we want to avoid it.

That such games can be played on chain at all, in today's Bitcoin, was shown by Poelstra [2]: each player signs the next game state with a one-time (Lamport) signature, which Script can read, and a script checks that successive states are related by the rules, so tic-tac-toe needs 27 pre-signed transactions rather than one per path through the game tree (about 250,000), and the same approach extends in principle to chess. His distinction between “Small Script”, expressive arithmetic on 32-bit values that cannot see the transaction, and “Big Script”, which can, through `OP_CHECKSIG`, is a useful lens for what follows: the predicates that judge a move here are Small Script over one-time-signed state, and only the payout is Big Script. What that

¹For those readers who balk at this characterization, note that the author has been paying his utility bills for years, and along with thousands of other payments during that time with near zero payment failures, on self-custodial Lightning, and for practical, not ideological, reasons, far prefers using LN as a payment technology over any existing fiat rail.

work leaves open is the one-move-per-transaction pace of play on chain, which is the subject of the rest of this section.

One approach to doing this offchain is called ForceMove [3]. The idea is relatively simple: behave optimistically, and fallback to onchain transactions as the arbiter if your opponent does not follow the rules you agreed to. The reason for its name is that, once on-chain, the opponent is *forced to move* by having a timeout expressed in the blockchain’s consensus logic. So in Bitcoin this would mean that the Script controlling the funds has a shape like:

$$<T_1> \text{ OP_CHECKSEQUENCEVERIFY OP_DROP } <P_A> \text{ OP_CHECKSIG,}$$

where P_A is the key of the counterparty not on-move, i.e. they win by default if their opponent doesn’t move.

This seems to be the right general paradigm: if you’re forced to move, the game cannot stall, but your ‘move’ can be of a type constructed to make it easy to prove fraud whenever the move is illegal. (I haven’t bothered to note how terminal states are resolved, i.e. win/lose/draw; that’s less important and pretty simple, usually).

2.1.1 ForceMove is not the full solution

ForceMove is clearly “correct”; there is no way to have the game be fair without such a timeout clause. So if you lose the ability to publish within a time window, you forfeit the game. Note that this is exactly the same tradeoff that Lightning itself makes for the case of simple payments.

But the problem with ForceMove, which is not exposed in the Lightning payments case, is that the amount of interaction forced on-chain is potentially huge, depending on the game. Consider this scenario:

- Alice and Bob agree to play chess offchain, set up a contract in a Lightning channel (details deferred till later)
- Alice makes move 1 as a message offchain, both sides cosign and state is updated; it’s Bob’s move.
- Bob stalls, does not send his move
- Alice is forced to resolve onchain: sends the state in a transaction, the output has a timeout forcing Bob to respond
- Bob *does* respond, with a valid move ...

...now Alice is stuck playing the game onchain; a chess game can easily take 50-100 moves by both sides. If Bob is adversarial he will know that Alice will not want either the transaction cost nor especially the delay implicit in doing all of those moves onchain (she intended to play offchain, which can be near immediate), so he could bribe her. This would be much more relevant if the game is really financially motivated (\$ 1000 bet), and in case where we are at move 30 and Bob knows for sure he’s losing; instead of resigning he says to Alice “here, I’m generous, take \$ 800 and I’ll take \$ 200, otherwise, we’re going to chain”.

The point is not that that scenario is particularly realistic, the point is that **going onchain is a viable threat for an adversary**, in general, and that’s why I say “ForceMove is not the full solution”.

2.1.2 Can we do ForceMove offchain?

The next natural step. If we want to enforce a timeout clause like the above, but *offchain*, then clearly putting it into a Bitcoin Script is possible, but that returns us precisely to where we started: ForceMove’s forced move is only effected on-chain in the adversarial case; the thing we were trying to avoid.

There’s a nuance here worth being precise about:

- If one side forces on-chain, publishing a dispute transaction, and says “the other guy didn’t follow the rules! here’s proof”, with a signed attestation by the counterparty of an illegal move, no problem: the court can adjudicate the dispute.
- If one side forces on-chain, publishing a dispute transaction, and says “the other guy didn’t send his move”, that’s very different. How does the “court” (blockchain) know whether the move was sent or not? Clearly that’s impossible, so the court has to treat both sides fairly, and insist they play out their moves. We have reverted to exactly the same problem we had in vanilla ForceMove.

From this reasoning it’s clear: to solve the problem of that last case, we need an objective fact that the court can assess, not hearsay. That objective fact must take the form “by the count of some external attestation, the counterparty did not post a move before the deadline”. In the next section we’ll examine two different ways to create that external attestation.

The following diagram (Figure 1) shows at a high-level how this logic can be enforced in a single “contract output” embedded in a Lightning channel. The most important point is that “the counterparty did not move before the deadline” is being attested to by an external “venue”.

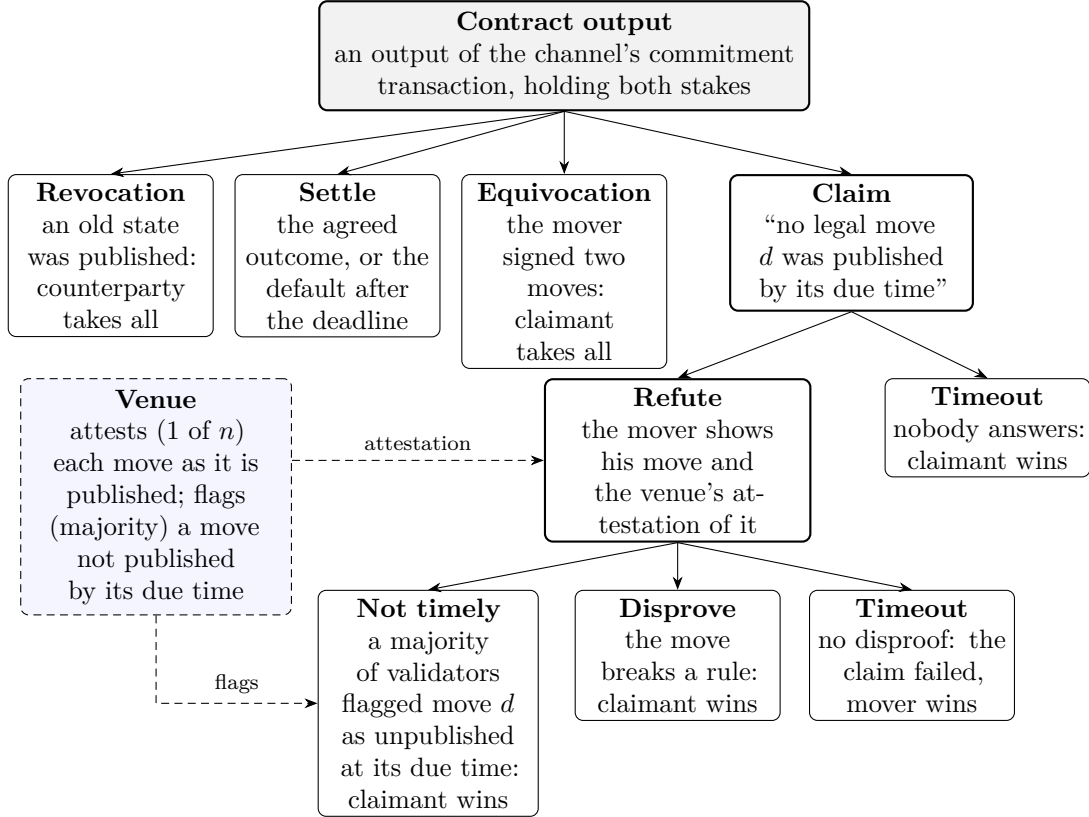


Figure 1: The dispute logic of one contract output, at a high level. Only the claim and its answers involve the venue: a claim of absence is answered by the venue’s attestation that the move was published, and that answer is in turn answered either by the game’s rules (the move was illegal) or by the venue again (the move was published too late). Every branch that pays someone is a Bitcoin transaction prepared in advance; the arrows are spending conditions, not messages.

2.2. From a proof of publication ledger to a venue

The design began, as this section does, from the idea of a separate proof of publication ledger, and the options below are ledgers. It turned out that a ledger is more than a two-party contract needs. No order across moves or across contracts is used, and no reader other than the two players ever consults a contract’s record, so no history needs to be unique: what each move needs is that it was made available, and a decision whether that happened by its due time. Uniqueness of a move comes from the mover’s own one-time key, not from the record. We therefore call the service a *venue*, and the combination of publication with deadlines a *public clock* (Section 6); a committee can be drawn per contract (Section 12.2), and uniqueness in the sense of a single-use seal is needed only for facts that many contracts share (Section 6.2).

Bitcoin itself gives proof of publication, for data placed on chain, where every node sees it. Its better-known relative is proof of existence, exemplified in the simple, but useful, OpenTimes-tamps project [6], which commits only a hash: it shows that someone held the data at a time, not that anyone else saw it. As discussed earlier, it’s not practical to use this to adjudicate

onchain, because Bitcoin SPV proofs in Script are not practical. If we make a separate one, we could use proof of work as a consensus mechanism for deciding a canonical chain, or we could use a set of bonded validators, where the security assumption is: if members of the quorum equivocate on the state, their bond can be slashed. This is the equivalent economic security assumption of the reorg threat in the proof of work variant. Let's consider PoW and bonded validator².

2.2.1 Proof of Work

As already noted, proving inclusion in Bitcoin's PoW chain with its SHA2 linked chain of headers is not practical in Bitcoin Script, without new opcodes (for example CAT for Merkle trees; we defer the details to many other papers and discussions). But since we are designing from scratch, we are not limited to that.

Considering that arithmetic in Bitcoin Script is more or less limited to what Poelstra calls "Small Script" [2], which operates on 4 byte numbers, what we can do is design the proof of work algorithm to use nibble arithmetic only. This is realized in our sample code, in the `n4bit` crate of the proof of concept [23]. Note that there had already been a very similar construction made in 2025 by AbdelStark, see StackSat-128 [17]. So in general terms, imagine the proof of work hashing uses this nibble based algo, such that proving a hash chain is correct in Bitcoin Script is actually feasible. This would mean a contract participant can post something to the proof of publication ledger, and if their counterparty challenges, saying "you didn't publish anything", they can prove that in fact they did, in Script, thus winning the dispute over the pot.

Consider reorgs and canonical history: the key feature of proof of work is that the canonical history is defined by the heaviest chain. So imagine: Alice says "Bob didn't publish his move", then Bob says "yes I did: it's in this PoP ledger block at this height, and here's the header chain back to genesis/checkpoint". Alice can counter-refute: "no you didn't: here's an alternate header chain with more weight, proving that yours is not canonical". This is of course both necessary but also shows the fragility: necessary because otherwise Bob can "mine" a super-cheap chain that costs almost no PoW, fragile because without the kind of enormous weight of something like Bitcoin's PoW chain, reorgs could be easy and frequent even without adversarial behavior.

The least desirable aspect of the PoW option is in there: it's that this costs significant resources to maintain, otherwise it offers no security at all. How are miners compensated for this work? Merge mining seems not to apply straightforwardly, but the biggest count against this idea is the following:

There is not a simple transaction fee paying process, since all contracts are settled in Lightning channels (one can concoct something here, but it's not simple); the contract participants can, by the very nature of proof of work's lottery, never know in advance who the miner will be. If the ledger is not a blockchain with its own coin and own set of consensus rules, there is no viable way to create a positive incentive to include publications.

²We are not calling it proof of stake, because while the consensus algorithm may be similar, it's a notably different security construction than one in which the stake is the tokens of the ledger itself; here the "stake" is the bond *in Bitcoin*, a timelocked slashable-to-burn output on the Bitcoin blockchain itself.

2.2.2 Bonded validator quorum

The alternative is to have a quorum of bonded validators decide what was published.

It turns out that with careful thought, it's possible to construct a version of this that is both sufficiently efficient to enforce the necessary rules in Script, and also that has the right kind of both positive and negative incentives, purely measured in bitcoin, without inserting a new altcoin.

A naive idea of how to do this hits a snag: Bitcoin Script can read Winternitz signatures in a couple of kilobytes; weighty but OK. It cannot, directly, read Schnorr or BLS signatures on arbitrary messages (the former without CHECKSIGFROMSTACK, the latter .. well that would be a much bigger consensus update!). An aggregate signature (FROST, BLS) would let a whole quorum be checked at once, but it cannot say which members signed; and to assess 66 out of 100 Winternitz signatures in a Bitcoin script seems patently absurd.

Both problems go away once we look at what the validators actually have to say. The quorum never has to sign the *content* of a move. One member attests the entry (the positive statement, 1 of n), and the quorum says only one bit per move: “not published by its due time”. A one-bit statement costs one key per member, a point whose secret the member reveals when the statement holds, and t of n such reveals are counted in Script with OP_CHECKSIGADD, each naming the member who made it. The threshold is a count of named one-bit signatures, not a threshold signature, so it is attributable by construction. (A hash preimage per member would serve equally well for the count.)

The content attestation, by the one member, must also be read by Script, and here too a hash-based one-time signature would suffice: a table of preimages per (chunk, value), revealing those for the entry's values. We use instead an elliptic-curve version of the same one-time idea, which we call EC-OTS (Section 4.6): the venue publishes an *anticipation point* per (chunk, value) and seals an entry by revealing the discrete logs of the points it selects, and Script checks possession of each revealed scalar with one OP_CHECKSIG. The reason for the curve is not Script but payment. Scalars and points add, so the attestation of a whole entry by a particular member collapses into a single point: the sum of the points the entry selects plus that member's proposer point, whose discrete log becomes known only when that member seals exactly that entry. That is an ordinary adaptor point, so the mover can pay the member for sealing, atomically, with a pre-signed transaction that the member's scalar completes (Section 4.7). This is the shape of a DLC, whose anticipation points exist so that transactions can be made conditional on an oracle's outcome; here the outcome is “this entry was sealed, by this member”, and the transaction is the member's fee. Hash preimages do not add, and a lock on a whole entry's worth of them would be a special output checking every one in Script, not a point a payment can be locked to.

What you need, then:

- **Liveness** : can never be promised, just like in a proof of work chain, but if *any* member of the validator set can enter your publication in a time window, then some approximate “1 of n liveness” can be claimed, and this is an acceptable security model.
- **A large enough quorum**: Credibility of the above as well as what follows, requires some relatively large, heterogeneous and ideally permissionless-entry set of validators. At one key per member per move, a quorum of 50 of 100 is practical in Script. Even figures 10

times higher than that are *theoretically* possible, but in practice we would not go much higher.³

- **Fraud is attributable:** this is why the quorum’s statements are one-bit signatures by named members and not a threshold signature. A FROST-style signature by its nature cannot reveal which members of the validator set committed the fraud, and punishing the entire set when only a threshold of it committed the fraud is clearly unworkable.
- **Positive incentives in bitcoin:** this is possible in a more general “sidechain” model (pay fees to move around coins on the sidechain), but this is not a sidechain, nor any kind of blockchain: the only thing the validators decide is whether each publication was made in time; the semantics of the publications are none of their business. Here we have validators post bonds on bitcoin (timelocked coins) and receive fees for their publication efforts, atomically with those publications, over Lightning channels, using the adaptor points above. This incentivizes honest participation and disincentivizes fraud or dishonesty.

You may be wondering why we insist on *attributable* fraud. Without attribution, users of the venue cannot discern which members they do not want to use in future, if something fraudulent is published. With attribution, a member caught contradicting itself is ejected: it loses its future fees while its bonded coins (coins locked to a CLTV output) sit out their timelock. Timelocked coins as a costly identity are not new: JoinMarket’s fidelity bonds [9] use exactly such outputs to make a Sybil attack on its market of coinjoin makers expensive, and are, to our knowledge, the only deployed use of timelocked bitcoin for that purpose; Section 12.2 returns to them for a roster that anyone can join. It’s possible to go further, as an option: equivocation evidence can be made to unlock the bonded coins to anyone, but without covenants this is not a simple burn, it is a miner fee-race burn: since the evidence is public, any miner can include its own spend paying the whole bond as fee, so the money game-theoretically ends up with the miner of the first eligible block, and the cheater recovers it only by mining that block itself. We do not make this the baseline (Section 3).

2.3. What the venue attests to

We’ll continue to use the two simple motivating examples: tic-tac-toe, and chess. We’ll discuss external facts, i.e. something not limited to a 2 player game, in Section 6.

What would an attestation look like? Probably a variant of:

$$(S, m, S', t)$$

where we set a notation of S for the game’s internal state (e.g. chess: piece positions on the 64 square board plus a couple other details like castling), m for the move being presented, and t an optional extra defining “terminal” state: undefined, win A, win B, draw etc. depending, again, on the game.

Specific cases *may* justify condensing this; the most natural one might be removing S , the state *before* the player’s move.

³Section 12 returns to this: a large, dynamic set need not be verified in Script at all, if its role is to make a small, Script-legible committee accountable.

Each player attests to a move choice, of course; but there’s a subtlety that affects the question of how condensed the game state S attested to should be. In the case of tictactoe it can be as few as 20 or so bits; in the case of chess it can be 35 bytes.

The subtlety is that the court sees nothing but what the dispute puts in front of it. When the claimant disputes a move, the only facts available in Script are the venue’s attestations of two consecutive entries: the one at the disputed depth and the one before it. So the attested state must be *self-sufficient*: everything the rules need in order to judge a move must be recoverable from the pair (S, S') plus the move itself, with no appeal to the rest of the game’s history. For tic-tac-toe this is trivial (the board and whose turn it is). For chess it means the state carries castling rights, the en-passant square, the side to move and the halfmove clock (but no status field: a mated side simply has no legal move, so the game ends by resignation or a stall), and it rules out adjudicating rules that need the whole history (threefold repetition) unless a digest of that history is itself part of the state.

Once the state is self-sufficient, the natural condensation follows: an entry need carry only (m, S', t) , since the previous depth’s entry already carries S . The court reads both entries out and judges the triple (S, m, S') . This is the form we use: each entry’s *head* (48 bytes) holds the new state and the move (and, for tic-tac-toe, a status), the region the mover signs; the entry carries the mover’s one-time signature over that region beside the head; and a dispute parks the heads at depths $d - 1$ and d . The mover’s signature matters as much as the venue’s: a venue attestation proves an entry was *published*, the mover’s one-time signature proves the mover *made* it, and a refutation needs both for the new head, or anyone could answer a claim with any junk the venue happened to seal. For the prior head the claimant’s signature is enough: it pins the claimant’s own move $d - 1$, and whether that move was published is a different question (Section 4.3).

What the court needs from the venue is therefore exactly three kinds of statement per move of each contract, and nothing about the game:

1. *this* entry is move d of contract c (a positive attestation, by whichever member sealed it);
2. the member who sealed it was member i (so that it can be paid, and blamed);
3. move d was not published by its due time (a negative statement, by a majority of members).

Section 3 describes how the venue produces these, and Section 4 how a Bitcoin script consumes them.

Order comes from the heads. A venue that keeps no order (Section 3) may seem to invite trouble: what if Bob publishes move 4 before Alice has published move 3? Nothing breaks. The venue seals it, since it checks only that the right mover signed an entry for that move, and the seal is timely, being early. But Bob’s head carries the whole position after move 4, so it commits him to one particular move 3 by Alice. If she plays it, the game continues and Bob has only shown her his reply in advance; if she plays anything else, his move 4 no longer follows from her move 3 and the disprove family refutes it; and he cannot replace it, since his one-time key for move 4 is spent and a second move 4 is an equivocation. An early move is a binding, conditional commitment that can hurt only its maker. (A correspondence player once answered 1.e4 with “1...b6, and 2...Bb7 whatever you play”; his opponent replied 2.Ba6, and the pre-committed

bishop move lost a piece.) This rests on three properties of the contract, not of the venue: every move number has one designated mover, so an early move takes no one else’s turn; every head carries the state it follows from, so an early move pins down the history it answers; and every move number has its own one-time key, so an early move cannot be withdrawn. A rule that needs order the heads cannot express, such as “valid only if made after time T ” or “first to post wins”, is outside the design: the venue attests *by* a time, never *after* one, and simultaneous moves must be recast as commitments and reveals on alternate move numbers.

3. The venue

The venue is a set of n bonded members that attest the moves of the contracts registered with it. It is not a blockchain: it has no coin, executes nothing, keeps no global order and no sequence of slots (the order of a contract’s moves is carried by their heads, Section 2.3). It answers, per move of each contract, the two questions a dispute asks: *what* was published, and *whether it was published in time*. The two answers come from different mechanisms. In the terms of Discreet Log Contracts [24], the venue is a federation of oracles: each move’s content is an outcome attested by one member under anticipation points published in advance, and each move’s timeliness is a one-bit outcome attested by a majority. What is new is not the attestation but what is attested (publication and, above all, absence), and that the attestations are read out in Script so that the game’s rules can judge the content.

Not a federation. Both this venue and a federated sidechain such as Liquid [10] rest on a majority, but not on the same one. A federation holds the coins: its keys sign every withdrawal, so its majority can move any funds, and can freeze them. The venue holds no coins and decides one bit per move, whether it was published in time; it cannot forge a move, which the player signs, nor change a rule, which Script enforces, nor stop anyone leaving, since every user’s funds sit in its own channel and a unilateral close is always available. A federation’s majority can take or freeze everything in its custody, on its own; the venue’s can at worst tip one disputed contract, by misstating the time of a move, for one party to it and against the other. Section 11 develops the comparison.

The proof of concept has one fixed set of members; Section 12.2 proposes drawing each contract’s members from a larger bonded roster instead.

3.1. What: attestations keyed by contract and move

A contract c registers with the venue for moves $d = 1..M$, and the venue publishes, for every (c, d) , a *content table*: under the venue’s *content key*, a Schnorr key X that every member holds, one anticipation point $A_{(c,d),j,v}$ per header chunk j and nibble value v . Sealing move d of contract c is attesting its entry under that table: revealing the discrete logs of the points the entry’s header selects. The attestation is an EC-OTS; Section 4.6 gives the construction. Its property for our purposes is that a Bitcoin script can check, one OP_CHECKSIG per chunk, that the venue attested a particular value at a particular position, with no signature verification over arbitrary messages. Because the table is keyed by the move and not by any schedule of the venue’s, many contracts share one venue, each with its own timing.

Because the content key is shared, *any* member can seal any move. Each move has a *designated* sealer by rotation, $p(c, d) = (h(c) + d) \bmod n$ with h a hash of the contract’s identifier; the mover

submits its signed entry to that member and, if no seal appears within a backoff b (seconds, covering propagation), to the next member in the rotation, and so on. Inclusion therefore needs only one honest, live member within the move’s time (“1 of n liveness”). A shared key cannot say who used it, so each member also holds, per move, a *proposer* key $P_{i,(c,d)} = p_{i,(c,d)}G$ and reveals $p_{i,(c,d)}$ in the seal it makes. Every refutation on Bitcoin names the member whose seal it relies on (Section 4.3).

A seal counts only if it reaches the members before the move’s due time, since they flag what they have not seen. Seals are public and self-authenticating, so the mover publishes every seal it receives to all members (and to any audit layer, Section 12): a sealer that reveals its seal only to the mover is defeated by the mover’s own dissemination. Near the deadline the mover may also submit to several members in parallel, which is safe because seals of one entry are byte-identical and differ only in their proposer reveal: a sealer that delays its seal until the last moment is defeated by the others. Withholding or delaying therefore works only if every member does it, which is the 1-of- n assumption again.

Honest members seal only entries the mover signed: the contract’s registration carries the movers’ per-move state keys, and a member checks the signature (never the move’s legality, which is the disprove family’s business). A member that seals an unsigned entry is caught by the entry itself, and ejected; the seal is inert in any contract, since a refutation must carry the mover’s signature.

Two seals of the same move are therefore one of four things: the same entry sealed by two members (byte-identical attestations: the shared key and the per-value nonces are deterministic); two different entries both signed by the mover (the *mover’s* equivocation, answered by the contract’s **equiv** leaf); an unsigned entry (the sealer’s fault, provable); or a seal made after the due time (next). None needs a canonical choice between them, so the venue needs no fork-choice rule.

3.2. When: due times and flags

Each contract fixes its timetable at open: move d is due at $T_d = t_0 + d\ell$ (seconds per move ℓ ; other rules, up to a chess clock, are possible, since only the members read them). Members judge “due” against their own clocks, which need agree only to within ordinary clock skew. The negative statement is a one-bit signature: each member holds, per move, a *flag* key $F_{i,(c,d)} = f_{i,(c,d)}G$ and reveals $f_{i,(c,d)}$ if at T_d it held no attested, mover-signed entry for the move. A flag is a single point whose secret is revealed or not, so a script counts flags with **OP_CHECKSIGADD** at the cost of one key per member. A refutation relying on a seal that t members flagged is killed by the **not_timely** leaf. The threshold is a bare majority, $t = \lceil (n+1)/2 \rceil$: a flag is one-sided (false absence costs t named flaggers; a late seal left standing costs its sealer plus the silence of $n-t+1$ honest members), and the two failure costs balance at the majority.

This is what makes timeliness enforceable against a colluding member. Honest members do not seal a move after its due time; a member that does, for a stalling player, produces a perfectly valid attestation, and without flags a refutation built on it would defeat an honest absence claim. With flags, the honest majority’s on-time statement “move d was not published” outweighs the late seal.

Bitcoin’s clock is read in one place: the earliest time an absence claim for move d may be broadcast, an absolute timelock at $T_d + m$ against median-time-past, the margin m covering

the lag of median-time-past and a miner’s freedom to skew timestamps. All other windows in a dispute are relative timelocks, where block granularity only slows a resolution. The per-move clock runs at the venue’s pace; Bitcoin’s clock measures only the $O(1)$ dispute about it.

3.3. A better-conditioned oracle problem

The venue is a federation of DLC oracles, but it attests a different kind of fact from the one DLCs are usually imagined for, a price at maturity, and the difference matters for security. The key property is that the fact concerns the *action of the party who benefits from it*. A mover’s entry, posted in an `OP_RETURN`, proves that the mover published it by that block: the mover signed it and Bitcoin dates it. A price posted to Bitcoin proves only that someone claimed that price. Publication is *self-certifying*; a price is not, since nobody can produce evidence of “the price at noon”, only report it.

Three things follow in the venue’s favour. The honest party can strengthen the truth: a mover disseminates its seal to every member, to an audit layer or to Bitcoin (Section 12), so a lie must override evidence that is widely held, while a party wronged by a price oracle can do nothing to make the true price more evident. The reference cannot be manipulated: a price DLC can be attacked without touching its oracle, by moving a thin market at the fixing moment, while the venue’s “underlying” is the mover’s own action, and the only analogue is an attack on the network. And the oracle decides less: a price attestation *is* the payout, while the venue attests only publication and timing, and the game’s rules judge the content cryptographically.

The price for this is paid elsewhere. A price oracle attests one event for every contract that uses it, so a lie in one party’s favour is told to the whole world, visibly; the venue attests per contract, so a colluding majority can act against one contract silently. Submission is private between mover and member, so a false flag is attributable but, without a public record, not provably false, where a blatant price lie is evident to all. And a price DLC needs its oracle once, at maturity, while a game needs the venue at every move. The self-certifying property also covers only the positive fact: for absence, nobody can produce evidence unless every entry goes to a public record, and the venue’s trust concentrates there.

	price-feed DLC	the venue’s per-move attestation
can the interested party produce proof?	no	yes, for publication (not for absence)
manipulating the underlying	a real attack	not applicable, apart from network attacks
what the oracle decides	the payout directly	timing only; content is cryptographic
can a lie be aimed at one contract?	no: it is broadcast	yes, silently (mitigated by sampling, Section 12.2)
publicly checkable by default	yes, for blatant lies	no; needs an audit layer (Section 12.5)
liveness needed	once, at maturity	at every move

At slow timescales the two converge: a daily closing price across many exchanges and a move visible to an audit layer within the hour are both effectively objective, and at fast timescales both blur, through manipulation or through propagation. Objectivity alone therefore does not

separate them; who can produce the evidence does. The shared facts of Section 6.2 are of the price kind, and inherit its profile: broadcast and untargetable, but with no party able to prove them.

3.4. The registry

Everything a contract needs from the venue is fixed at open. The contract’s *registry* lists the members, their proposer and flag points for every move, and the content tables, each co-signed by every member; the contract pins it by root. Members see a contract’s identifier, timetable and movers’ state keys, never its content: they publish points in advance and scalars when they seal or flag, and whoever disputes uses them. Contract identifiers must be unique on a venue (two contracts sharing one would share tables), and registration is paid, since it costs the venue a table’s curve arithmetic per move: about 100 KB of points and 100 ms of computation per move, so some 10 MB and 10 s for a contract of a hundred moves. Tables cannot be shared between contracts (the one-time property forbids it); attesting only the 48-byte head rather than the whole header would halve the cost.

3.5. Incentives

Fees. A mover pays the member that seals its move, atomically, with a point-locked payment inside the mover’s channel. The lock point is $T = C_h + P_{i,(c,d)}$, the sum of the content points the mover’s head selects and the intended sealer’s proposer point; its discrete log is revealed exactly when member i seals exactly that head as that move (Section 4.7). Publication and payment are one event, as a preimage and a payment are one event in Lightning.

The designated sealer is generally not the payer’s channel counterparty, so the fee must be routed: the payer’s hub (for instance a Lightning service provider coordinating fee payment to the venue) offers the same lock onward, to member i or to the next hop, locked to the same point T (less a routing fee), and when member i seals, the revealed t settles each hop in turn, backwards. This is a *point-time-locked* contract (PTLC) route, not a hash-locked one: t is a discrete logarithm, the sum of the attestation’s scalars and the sealer’s proposer scalar, and no hash preimage can stand for “the venue attested exactly this head”. The payment is held only from submission until the seal, seconds, so it has the shape of a hold invoice without the liquidity cost of a long hold. PTLCs are not yet deployed on the Lightning Network; the scheme relies on them, or on channels that implement the point lock directly, as ours do (the lock is an ordinary contract output with an adaptor pre-signature, Section 4.7). The proof of concept builds the single-hop case in its test suite: the lock sits in the payer’s channel with a hub that also runs the sealing member (the browser demonstrations charge no fees). Sealing is not contingent on the fee arriving: the lock is the member’s reward, so a payer’s route (in that test, through the opponent) cannot be used to censor. This is a rule, not an incentive: a member whose fee is blocked may rationally decline to seal, and 1-of- n liveness assumes that members honour the rule, their motive being the venue’s franchise as a whole. Fees should therefore not be routed through the payer’s opponent.

Bonds. Each member locks bitcoin in a timelocked output on Bitcoin for the duration of its membership. The lock is a sacrifice of time value, not a slashable deposit in the baseline design: a member caught contradicting itself (an unsigned seal; a lock completed without the matching seal; a seal against its own flag) is ejected, losing its future fee stream while its lock sits out its

term. The lock is not a second penalty beside the lost fees: the coins sit out their term whether or not the member cheats, so their time value is the price of the franchise, paid on entry, and what ejection takes is the franchise itself. A venue may additionally attach evidence leaves to the bond, spendable by anyone holding a proof of equivocation; without covenants such a spend is a fee race, so the bond ends as miner fee, a burn in effect.

The two penalties suit two different kinds of fault. A *self-contradiction* (two statements under the member’s own keys that cannot both be honest) is provable without reference to time, and an honest member never produces one as long as it keeps its state: it records a seal before releasing it, never flags a move it sealed, and claims a fee only after its seal is out. A member that loses that state and contradicts itself is in the position of a Lightning node that loses its channel state and broadcasts a revoked one, and the same answer applies: keeping state is the operator’s duty. Only self-contradictions are fit for burning. A *timing fault* (a flag of a move that was in fact published, a seal the record shows as late) depends on clocks and propagation, and an honest member can commit one through skew or a delayed message. It is at most grounds for ejection, and only outside a tolerance for skew and propagation (Section 12). Scaling a penalty with how many members fault together, as Ethereum’s proof of stake does [25], would separate isolated accidents from coordinated theft; we leave it as a direction for future work.

The resulting condition. An equivocation that needs t members is irrational when

$$t \cdot \frac{\text{PV}(F)}{n} > G,$$

where $\text{PV}(F)$ is the present value of the venue’s total fee stream over the members’ remaining terms and G the value the equivocation can extract. The lock enters as a bound, not a term: a member locks only if it expects its fees to repay the time value L of its lock, so $\text{PV}(F)/n \geq L$ for members who joined, and L can be read off the chain by anyone; competition for membership pushes the two together, as Linus argues for Stakechains [8]. Only the optional evidence leaves, which burn the principal, add a loss of a different kind. G is an aggregate even for two-party games: a colluding majority can act against every contract it serves within one dispute window, and pays its franchise once. It needs a colluding player in each contract, and a hub, which is the counterparty of many contracts, is exactly such a player; so G is the value in flight across the venue, not one pot. For a fixed threshold fraction t/n the security budget is a fixed share of the venue’s fee revenue, whatever n is: as with Bitcoin’s miners, the number of validators buys decentralisation and liveness, the revenue buys safety. Section 11 takes this up, and Section 12.2 shows how per-contract committees reduce G for a targeted attack to about one pot.

4. Contract outputs and the dispute graph

This section gives the transaction-level view of a game contract, as implemented in the proof of concept [23]: what sits on each commitment transaction, which transactions are pre-signed when a channel state is agreed, which are built at dispute time by one party alone, and how the venue’s attestations enter the scripts as public-key constants. We use tic-tac-toe as the worked instance (a 48-byte entry head, at most $M = 9$ moves) and quote chess where the numbers differ. Throughout, A and B are the two players (A white, B black); each holds a payment key P_A , P_B , and every 2-of-2 leaf below is

$$\langle P_A \rangle \text{ OP_CHECKSIG } \langle P_B \rangle \text{ OP_CHECKSIGADD } \langle 2 \rangle \text{ OP_NUMEQUAL}$$

(or `OP_NUMEQUALVERIFY` when the leaf continues), with the witness carrying σ_B then σ_A . Time-locks: Δ is the channel’s `to_self_delay`, δ the challenge window and δ' the additional delay on prover-favourable paths; move d is due at $T_d = t_0 + d\ell$ and a depth- d absence claim is admissible once median-time-past has passed $T_d + m$ (an absolute, time-based `OP_CLTV`). We write s for the venue’s key index of move d of this contract, $s = (c, d)$ (§3).

4.1. Keys and venue data fixed at open

The two parties agree the contract at an off-chain draft. Beyond the channel keys, the draft fixes, per depth $d \in 1..M$ (move d is made by the *mover* $m(d)$, alternating from A at $d = 1$; the other party is the *claimant* $c(d)$):

- K_d^{ref} , the mover’s *refute key*: a Winternitz one-time key ($w = 16$, `HASH160`) over the pair of venue heads $h_{d-1} \parallel h_d$ (96 bytes, 192 message digits plus 3 checksum digits; 48 bytes at $d = 1$, where the prior state is the constant empty board);
- K_d^{st} , the mover’s *state key*: a Winternitz key over the signed region of the entry (the 3 state bytes for tic-tac-toe; 42 bytes for chess). The venue entry the mover publishes carries this signature inline, so anyone can check authorship of a published move without the mover’s cooperation;
- K_d^{code} (mover) and K_d^{ccode} (claimant): Lamport keys over the 2-bit outcome code $X \in \{\text{A wins, B wins, Draw}\}$.

From the venue’s registry for this contract the draft pins, per move d (key index $s = (c, d)$):

- the *content table* $A_{s,j,v}$, $j \in 0..191$, $v \in 0..15$: one anticipation point per header chunk per nibble value, all under the venue’s shared content key (§3);
- the n members’ *proposer points* $P_{i,s}$ and *flag points* $F_{i,s}$, $i \in 1..n$, and the flag threshold $t = \lceil (n+1)/2 \rceil$.

Only the 96 head chunks of each table ($j \in 80..175$, the entry head inside the 96-byte header) are ever embedded in a script; a refutation reads the head out, not the whole header.

4.2. The commitment transaction

Each channel state k has two commitment transactions, one per broadcaster, spending the funding output. Beyond `to_local` and `to_remote`, the commitment carries one output per open *contract*: the game output C of value V (both stakes), and, while a move’s attestation fee is in flight, a *fee lock* output L (§4.7). Every contract output is a taproot output with an unspendable internal key and a tree of named leaves. Like `to_local`, each contract output (C , and L while it exists) carries the standard revocation leaf beside its own

`revoke:` `OP_HASH160` $\langle h_{\text{rev}} \rangle$ `OP_EQUALVERIFY` $\langle P_{\text{cp}} \rangle$ `OP_CHECKSIG`

where h_{rev} is the broadcaster’s per-state revocation hash and P_{cp} the counterparty’s payment key, so a revoked state is swept in full by the counterparty exactly as in Lightning. Every leaf of C that the broadcaster can spend carries an extra $\langle \Delta \rangle$ `OP_CSV` `OP_DROP`, so a revoked commitment’s contract output is swept before anything can hang off it; the outputs further

down the graph (A_d, P_d) therefore need no revocation leaf of their own. We omit that prefix below except where it changes the argument.

Moves are not channel updates. The contract output and its whole graph, for every depth, are signed when the contract is added to the channel. A move is an entry signed by the mover and sealed by the venue; the opponent signs nothing, and the latest commitment already enforces every later move, since the graph reads the moves from the venue. Channel updates happen only to open the contract, to settle it (a fold, an agreed result), and for fee locks, and each new commitment carries C unchanged with its graph re-signed. So a player cannot stall a game by refusing to cosign: the only update near a move is the fee lock, and it is not routed through the opponent (Section 3).

The game output's tree is

C : **revoke**, **settle**, **absent_d** ($d \in 1..M$), **exhibit_d** ($d \in 5..M$), **equiv_d** ($d \in 1..M$).

settle:	$\langle \text{deadline} \rangle$ OP_CLTV OP_DROP $\langle \Delta \rangle$ OP_CSV OP_DROP 2-of-2
absent_d:	$\langle T_d + m \rangle$ OP_CLTV OP_DROP 2-of-2-VERIFY OP_1
exhibit_d:	$\langle T_d + m \rangle$ OP_CLTV OP_DROP 2-of-2-VERIFY proposer fragment two-head readout tied to K_d^{ref} , gated on "terminal"
equiv_d:	2-of-2-VERIFY WOTS-VERIFY(K_d^{st}) park digits WOTS-VERIFY(K_d^{st}) restore some digit differs OP_VERIFY OP_1

settle pays $R(\text{empty board})$ after the contract's deadline and is the fallback when nobody claims. **absent_d** is the claimant's *thin absence claim*: "the mover published no valid move d by its due time". It asserts nothing about venue content, which is what keeps it two hundred bytes; the venue enters only if the mover answers. **exhibit_d** (tic-tac-toe only; chess needs none, because a mated side simply has no move to refute with) lets the mover of a terminal move park the attested pair directly and split by its result. **equiv_d** pays the non-mover the whole pot on exhibiting two distinct valid signatures under K_d^{st} : a Winternitz signature is forced by its message, so two signatures of different states are a proof that the mover double-signed at that depth, and no venue data is needed.

4.3. The claim output and the refutation

absent_d is a pre-signed transaction whose single output A_d carries the claim tree

A_d : **refute**, **counter** ($d \geq 2$), **split_X** ($X \in \{A, B, D\} : A_{\text{wins}}, B_{\text{wins}}, \text{draw}$).

split_X:	$\langle \delta \rangle$ OP_CSV OP_DROP 2-of-2-VERIFY LAMPOR-EXPECT(K_d^{ccode} , X) OP_1
counter:	2-of-2-VERIFY OP_1
refute:	$\langle P_{m(d)} \rangle$ OP_CHECKSIGVERIFY $\langle P_{1,d} \rangle \dots \langle P_{n,d} \rangle \langle n \rangle$ OP_PICK $\langle n-1 \rangle$ OP_SWAP OP_SUB OP_PICK $\langle n+2 \rangle$ OP_ROLL OP_SWAP OP_CHECKSIGVERIFY drop $n+1$ WOTS-VERIFY(K_d^{ref}) 192 digits on the stack AUTHORSHIP(K_d^{st} over the new head's digits) the mover's

AUTHORSHIP(K_{d-1}^{st} over the prior head's digits)	the claimant's
OP_TOALTSTACK $\times 96$	OP_2DROP $\times 48$
READOUT($A_{d,j},, j \in 80..175$)	new head only
OP_1	

The claimant's **split_X** is the timeout: after δ blocks with no refutation, the claimant reveals its code key to the outcome it asserts and both pre-signed signatures pay the corresponding distribution. The mover's **refute** is the only leaf that touches the venue, and it does three things in one script:

1. it names the proposer: the witness supplies an index i and a signature under $P_{i,d}$, made with the scalar $p_{i,d}$ that member i revealed in its seal (§4.6);
2. it *parks* the two heads: the WOTS verification of K_d^{ref} leaves the 192 message digits (the nibbles of $h_{d-1}||h_d$) on the stack, and two tied authorship checks prove that the mover's state key signed the new head's signed region, so a garbage-signed entry cannot be refuted with, and that the *claimant's* state key for depth $d - 1$ signed the prior's. That key is one-time, so only one prior can carry it (two would be the claimant's equivocation), and the mover takes the signature from the claimant's published entry. The prior is thereby the claimant's own move $d - 1$, which is all the disprove family needs of it; its digits are then dropped;
3. it *reads out* the venue for the new head: for each head chunk j the script pushes the sixteen points $A_{s,j,0..15}$, selects the one indexed by the parked digit, and requires a signature under it. Since the digit *is* the parked value, the readout ties “what the venue attested” to “what the mover re-committed” by construction, one witness element per chunk.

The per-chunk readout fragment, with the parked digit coming off the altstack, is

OP_FROMALTSTACK	$\langle A_{s,j,0} \rangle \dots \langle A_{s,j,15} \rangle$
OP_16 OP_PICK	OP_15 OP_SWAP OP_SUB OP_PICK
	select A_{s,j,v_j}
OP_18 OP_ROLL	OP_SWAP OP_CHECKSIGVERIFY
	possession of its secret
OP_2DROP $\times 8$	OP_DROP

16×33 bytes of constants and 20 opcodes per chunk; 96 chunks per head. A wrong digit selects a point whose secret nobody holds, and the leaf aborts. Only the new head is read out because the claim asks only whether move d was published; whether move $d - 1$ was is the counter's question, below, and the counter's tree reads h_{d-1} out as its own new head. Binding the prior by signature rather than by a second readout needs each game's signed region to cover every field of the prior that its leaves read. It does for all four games here; the unsigned first word of a head is read only by **wrong_slot**, which pins it to a constant, so forging it only fires that leaf against the mover.

counter (from $d \geq 2$) is the mover's own thin claim one depth back, “the claimant did not move at $d - 1$ ”: the negation of the claim's dueness. Its output is the depth- $(d - 1)$ claim tree verbatim (without a counter of its own), so a false counter is refuted by the claimant's attested, signed head at $d - 1$ and judged by the ordinary family, while a true counter means the claim was never due. One level closes it. The absence claim is thin, so without the counter the only answer to a claim at a depth the game never reached would be a refutation, which the mover cannot make: a claimant who stopped playing at move 3 could wait and claim **absent**₉₉, and

win. With it, the mover answers “you did not make move 98”, which the claimant cannot refute. An honest claimant claims the first move that was not made, where the counter is false; at any later depth the claimant’s own missing move comes first, and the counter beats the claim. This is also why the counter’s output has no counter of its own: a claimant that could truly answer “you did not make move $d - 2$ ” should have claimed that depth itself.

4.4. The refuted output

refute is pre-signed too (the mover’s payment key gates it), so its output P_d is pinned to the *refuted tree*

$$P_d: \text{disprove}_\pi (\pi \in \Pi_d), \text{not_timely}, \text{split}_X.$$

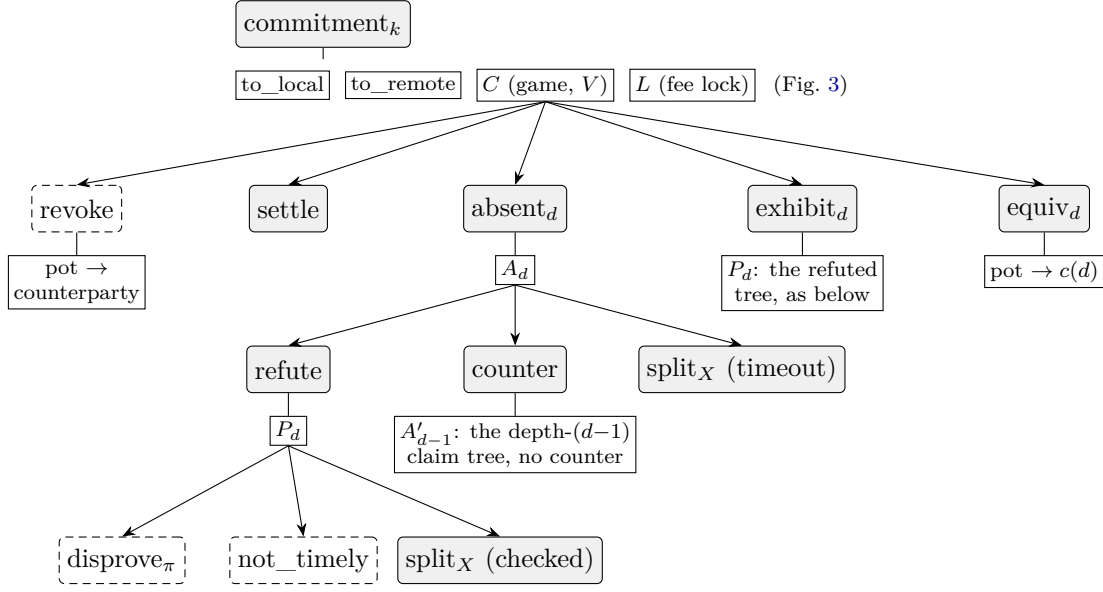
disprove$_\pi$:	$\langle \delta \rangle$ OP_CSV OP_DROP $\langle P_{c(d)} \rangle$ OP_CHECKSIGVERIFY WOTS-VERIFY(K_d^{ref}) PREDICATE(π over the 192 digits) OP_VERIFY OP_2DROP $\times 96$ OP_1
not_timely:	$\langle \delta \rangle$ OP_CSV OP_DROP $\langle P_{c(d)} \rangle$ OP_CHECKSIGVERIFY $\langle F_{1,d} \rangle$ OP_CHECKSIG $\langle F_{2,d} \rangle$ OP_CHECKSIGADD ... $\langle F_{n,d} \rangle$ OP_CHECKSIGADD $\langle t \rangle$ OP_GREATERTHANOREQUAL
split$_X$:	$\langle \delta + \delta' \rangle$ OP_CSV OP_DROP 2-of-2-VERIFY LAMPOR-EXPECT(K_d^{code} , X) WOTS-VERIFY(K_d^{ref}) RESOLUTION(new head) = X OP_VERIFY OP_2DROP $\times 96$ OP_1

The *disprove family* Π_d is the game’s rule set expressed as predicates over the parked tuple (prior state, move, new state): for tic-tac-toe **wrong_slot**, **cell_occupied $_i$** , **board_mismatch $_i$** , **turn_not_flipped**, **status_mismatch** and so on (about thirty leaves); for chess the twelve certificate kinds (a piece moving through an occupied square, a king left in check, ...) plus a well-formedness leaf, each taking a small exhibit (a square, an attacker) from the witness. Appendix A lists every chess predicate, with the layout of the 192 parked digits it reads, and follows one disproof (a bishop moving through a pawn) from the parked tuple to the spend. A disprove is not pre-signed: the claimant builds it at dispute time, copies the mover’s WOTS reveal from the confirmed refutation’s witness, signs with its own key, and takes the pot. **not_timely** is the timeliness disprove (§4.6): it counts signatures under the members’ flag points and kills a refutation whose move t members flagged as empty at its deadline. The mover’s **split $_X$** is *self-checking*: unlike the claim output’s timeout split, it must also re-park the pair and prove in script that $R(\text{parked new state}) = X$, because a refutation carries no adjudicated outcome for a Lightning-style code-mismatch leaf to judge. It fires only after $\delta + \delta'$, so every disprove has had its window.

4.5. Pre-signing

Every shaded transaction in Figure 2 is built and signed by both parties when a channel state is agreed, on both commitment versions, and its signatures are exchanged in the same message as the commitment signature. A script-path sighash commits to the leaf, so each leaf’s transaction needs its own pair of signatures. The counts are fixed by M :

$$1 + M(1 + 1 + 3 + 3) + M + (M - 1)(1 + 1 + 3 + 3) + \underbrace{4(M - 4)}_{\text{tic-tac-toe exhibits}},$$



shaded: pre-signed at each channel update (2-of-2, or the mover's key for **refute**);
dashed: built at dispute time by one party.

Figure 2: The game output C on one commitment and the transactions hanging off it, per depth d . Every output is a taproot tree; the edges are its leaves. The claim output A_d races the mover's refutation against the claimant's timeout; the refuted output P_d races the claimant's disproves against the mover's self-checking split; the counter sends the game one depth back. **revoke** is Lightning's revocation path, also a leaf of **to_local** and L (not drawn); **exhibit_d** is tic-tac-toe's only.

that is **settle**; per depth the claim, its refutation and the three-plus-three splits; the equivocation exhibit; per depth from 2 the counter with the previous depth's refutation and splits; and, for tic-tac-toe, per terminal depth the exhibit with its three splits. With $M = 9$ this is 166 transactions; for chess with $M = 8$ and no exhibit family, 129. The disproves and **not_timely** are not counted: they are the challenger's own transactions. Building the graph is dominated by hashing the refutation scripts (about 70 KB each, one per depth per version); signing 166 transactions is a few milliseconds.

4.6. How the points and signatures connect

The content attestation. The venue's content key is a Schnorr key $X = xG$ shared by its members. For key index $s = (c, d)$, chunk j and value v the venue derives a per-statement nonce $r_{s,j,v}$ and publishes the anticipation point

$$A_{s,j,v} = R_{s,j,v} + e_{s,j,v} X, \quad R_{s,j,v} = r_{s,j,v} G, \quad e_{s,j,v} = H(R_{s,j,v}, X, s \parallel j \parallel v).$$

Sealing a header $\mathbf{h}$ with nibbles $v_0..v_{191}$ as move s means revealing, for every j , the scalar

$$a_{s,j,v_j} = r_{s,j,v_j} + e_{s,j,v_j} x, \quad a_{s,j,v_j} G = A_{s,j,v_j},$$

that is, a Schnorr signature on the statement “chunk j of move s is v_j ” whose value is its own verification key. Scripts never verify these signatures; they verify possession of them. The mover proving that the venue attested h_d signs the refutation transaction itself with each revealed a_{d,j,v_j} as a private key, and the readout’s `OP_CHECKSIGVERIFY` under A_{d,j,v_j} succeeds exactly when the venue revealed that scalar. Revealing two values at one (s, j) is the venue equivocating; it is detectable by anyone holding both seals and is the object the flags and the proposer bit attribute.

The proposer bit and the flag. Content is a 96-nibble message and costs a table; a statement with no content costs one point. Each member i therefore holds two one-bit keys per move, $P_{i,s} = p_{i,s}G$ (“I sealed move s ”) and $F_{i,s} = f_{i,s}G$ (“move s was not published by its due time”), and reveals the scalar when the statement holds: $p_{i,s}$ in the seal it makes, $f_{i,s}$ as data. A revealed scalar is again a private key, so the disputing party signs its *own* transaction with it. In `refute` the proposer fragment selects $P_{i,d}$ by a witness index and demands a signature under it, so every refutation names the member that sealed the move. In `not_timely` the claimant signs under any t of the $F_{i,d}$ and leaves the rest of the witness slots empty; `OP_CHECKSIGADD` counts, an empty signature adds zero, and the leaf passes iff at least t members flagged. Members never see the contract or the transaction: they publish scalars, and whoever disputes uses them. The threshold is a majority because the flag is one-sided: false emptiness costs t named flaggers, while a late attestation left standing costs the proposer plus $n - t + 1$ silent members, and the two balance at $t = \lceil (n + 1)/2 \rceil$.

The parked tuple. The refute key is the bridge between the two halves of the script. A Winternitz signature under K_d^{ref} on the message $h_{d-1}||h_d$ reveals, for message digit i with value d_i , the chain element $\sigma_i = \text{HASH160}^{(d_i)}(\text{sk}_i)$, and the script checks $\text{HASH160}^{(15-d_i)}(\sigma_i) = \text{pk}_i$ digit by digit, with the usual checksum digits preventing a downward forgery. What remains on the stack afterwards is the 192 digits themselves. The readout consumes the new head’s as the selectors v_j ; the disprove predicates and the resolution check read all of them as the tuple. Since the key is one-time and the claim, exhibit and counter contexts are mutually exclusive spends of C , the mover can park at most one tuple per depth, and a disprove built from the confirmed refutation’s witness always judges a new head the venue attested, after the claimant’s own signed prior.

The fee lock. A mover pays the member that seals its move, atomically, with a point-locked payment in the channel (§4.7). Let $C_h = \sum_{j \in \text{head}} A_{s,j,v_j}$ be the sum of the points the head h selects and $c_h = \sum_j a_{s,j,v_j}$ its discrete log, which exists exactly when h is attested at s . Because the content key is shared, c_h alone does not say who sealed; the lock’s adaptor point adds the intended proposer’s bit:

$$T = C_h + P_{i,s}, \quad t = c_h + p_{i,s}, \quad tG = T.$$

The payer pre-signs the claim transaction for the payee with a BIP340 adaptor pre-signature: for message m (the claim’s sighash), nonce k with $R' = kG$ and challenge $e = H(R' + T, P_{\text{payer}}, m)$,

$$s' = k + e d_{\text{payer}}, \quad \text{verifiable as } s'G = R' + e P_{\text{payer}} \text{ given } T.$$

Only member i can complete it, $s = s' + t$, which is a valid signature under nonce point $R' + T$; and once the completed signature is public the payer extracts $t = s - s'$. The channel machinery sees only an ordinary point and an ordinary signature.

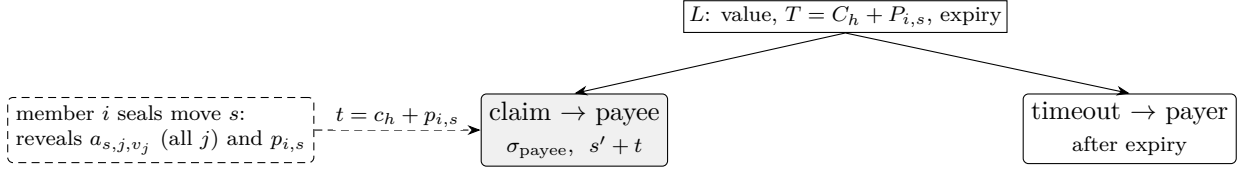


Figure 3: The fee lock output. The venue’s attestation of exactly the payer’s head as move s by member i reveals the scalar that completes the payer’s pre-signature; nothing else does.

4.7. The fee lock output

The fee lock L is a second, small contract output on the commitment while a move’s fee is in flight, with the shape of an HTLC:

```

revoke:    as above
claim:     [⟨Δ⟩ OP_CSV OP_DROP] 2-of-2  the payer's half: adaptor under  $T$ 
timeout:   ⟨expiry⟩ OP_CLTV OP_DROP [⟨Δ⟩ OP_CSV OP_DROP]
           ⟨ $P_{\text{payer}}$ ⟩ OP_CHECKSIG

```

claim is pre-signed to the payee’s address, the payee’s half a full signature and the payer’s the pre-signature above; it is broadcastable only once completed with t . **timeout** refunds the payer after **expiry**. Cooperatively nothing reaches the chain: the payee, holding t , hands it to the payer and proposes the next channel state with L removed and its value credited, and the payer’s policy accepts iff $tG = T$. A claim with no matching seal on the venue is answered by ejection rather than by script, and what names member i depends on the case. Claiming reveals only the sum $t = c_h + p_{i,(c,d)}$, and neither part can be found by search; but anyone can check $tG = T$, and producing t requires $p_{i,(c,d)}$. If i sealed nothing for this move, $p_{i,(c,d)}$ is known to i alone, so t by itself is a self-contradiction of member i against the venue’s record. If i sealed a different head for this move, $p_{i,(c,d)}$ is already public (the seal reveals it), and any member, who can compute content secrets, could complete a lock to another head; the evidence is then the claim’s provenance: its signature on Bitcoin, or, settled in the channel, a fact known to the payer. Naming i does not repay the payer, whose real loss is an unsealed move. Its protection is 1 of n : handed t with no seal, it takes its signed entry to another member within the move’s window, and it settles a fee only once the seal exists. When the sealer is not the payer’s counterparty, each hop of the route carries the same output, locked to the same T , with expiries decreasing towards the sealer as for HTLCs; the proof of concept builds the single-hop case.

4.8. Measured sizes

Table 1 gives virtual sizes, from the proof of concept’s test suite [23], as confirmed on a regtest node with real timelocks, for a five-member venue ($n = 5$, $t = 3$).

The shape of the costs is the point of the design. A stall, the common case, is an absence claim and a timeout: under 400 vB whatever the depth of the game. The venue is read out on Bitcoin only when a party answers a claim, and then once, in a transaction whose size is fixed by the header format and not by the game; the game’s own rules cost a few kilobytes per disprove, and a chess rule and a tic-tac-toe rule are the same shape of leaf. Every per-member statement of the venue is one point in a script and one signature in a witness, so the number of members

transaction	who	vB
commitment (two balances, one contract)	broadcaster	≈ 240
<code>absent_d</code>	claimant	188
<code>split_X</code> (timeout, off A_d)	claimant	193–201
<code>counter</code>	mover	178
<code>refute</code> , tic-tac-toe, $d \geq 2$	mover	20,073
<code>refute</code> , chess, $d \geq 2$	mover	24,104
<code>refute</code> , chess, $d = 1$ (one head)	mover	19,532
<code>disprove</code> , chess (one exhibit)	claimant	$\approx 4,950$
<code>not_timely</code> (3 of 5 flags)	claimant	254
<code>split_X</code> (checked, off P_d)	mover	$\approx 4,900$
<code>fee claim</code>	payee	171
<code>fee timeout</code>	payer	147

Table 1: Confirmed virtual sizes. A stall costs the claimant two small transactions at any depth; the refutation is the one large transaction and is paid by the party that answers a claim, its cost being mostly the venue readout of the new head (96 possession proofs, 16 points each, as script constants); the prior head is bound by the claimant’s signature instead, which costs about a quarter as much.

changes the graph by tens of bytes per leaf: going from 5 members to 50 adds about 1% to the chess refutation. The hard limit is the flag count: `not_timely` holds a witness slot per member on a stack of at most 1,000 elements, so a committee cannot exceed 999, and in practice about a hundred is the useful ceiling.

5. A game from open to close

Putting the pieces together for one game, and following the branches of Figure 1:

Open.

The players agree the contract in their channel: stakes, depth limit M , the registry root, and the one-time keys per depth. They sign the new commitment and the graph of Section 4.

Honest play.

Each move is an entry published on the venue in its time, with its fee paid through a fee lock; the move itself needs no channel update between the players. At the end, the loser folds: the channel is updated to the result and the contract output disappears. Nothing reaches Bitcoin.

A stall.

The mover publishes nothing for move d . After $T_d + m$ the claimant broadcasts the absence claim, then after δ the timeout split. Two small transactions, under 400 vB, at any depth.

A false stall claim.

The mover did publish. He answers with the refutation: the venue’s attestation of his entry, his own signature on it, the claimant’s signature on the previous move, and the name of the sealing member. The claimant has nothing to disprove; after $\delta + \delta'$ the mover’s checked split pays him.

An illegal move.

The mover published and signed an illegal move, and answers a claim with it. The claimant copies the mover’s re-commitment from the confirmed refutation’s witness and spends the one disprove leaf the move violates.

A late move.

A member colluding with the mover sealed his move after T_d . The honest majority flagged move d at T_d ; the claimant spends `not_timely` with t flags.

Equivocation by a player.

The mover signed two different moves at depth d (for instance, one to the venue and another to the counterparty). Two signatures under a one-time key at one depth are their own proof; the `equiv` leaf pays the claimant, with no venue data.

A claim at the wrong depth.

The claimant claims absence at a depth past the one where the game stopped. The mover counters, the claimant cannot refute the counter, and the timeout pays the mover.

A revoked state.

As in Lightning.

The asymmetry of cost is deliberate. The common failure, a stall, is cheap to prove. The expensive transaction, the refutation, is paid by the party answering a claim, and a party answers only when it will win. Winning does not, however, recover what the dispute cost: the pre-signed fees come out of the pot, so they fall on the winner. A player about to lose can therefore make a false claim for a few hundred virtual bytes and force the winner to pay for a refutation of some 24 kvB, and some hours of delay, at no further cost to itself. The dispute ends the contract, so this can be done once, not every move; but it is a lever for extortion, and it puts a floor under the pot below which an honest refutation does not pay.

Who pays for a dispute. Every on-chain dispute exists because someone deviated: stalled, refused to fold, claimed or countered falsely, or answered with an invalid or late move. If the contract is correct, the winner of any on-chain resolution is the party who did not deviate, so “the loser pays” is “the winner is reimbursed by the loser”. It suffices that each side adds a *dispute deposit* D to the contract output beside its stake. Cooperative settlement (a fold, an agreed result, a refresh) returns each deposit to its owner; an on-chain resolution pays the winner the whole output, as the splits already do, so the winner receives the loser’s deposit, and that deposit pays the dispute’s fees. An honest claim is never penalised: a real stall has no valid refutation, so the claimant wins and is reimbursed; the only ways to drive an honest claimant into an expensive path (an invalid refutation, a false counter) are deviations that lose to it. A false claim costs the honest mover nothing net, provided D covers the most expensive path it can be forced into, about 30 kvB for chess. The graph is unchanged; only the contract’s value and the cooperative rules change (and `settle`, the no-dispute fallback, returns each deposit). The proof of concept implements this; with each side depositing 10,000 sat, a false claim’s three hops of fees are paid from the claimant’s deposit and the honest mover receives both stakes and its own deposit back. The price is capital: pre-signed fees are fixed when the graph is signed, so they and D must be set at a pessimistic feerate, and both players lock it for the game’s duration, recovering it in the cooperative case. The pot floor becomes a capital cost rather than a loss,

and shrinking the refutation (most of it is the readout’s script constants) reduces it directly. A player about to lose can still delay the winner’s payout, at the price of its whole deposit.

6. Beyond games: a public clock and a seal

6.1. Two different services

The argument so far rested on one impossibility: between two parties, whether a message was delivered cannot be decided by anyone else. The receiver can always deny receipt, the sender can always claim to have sent, and there is no evidence of non-receipt; this is the fair-exchange impossibility without a trusted third party [4]. A contract therefore cannot punish silence unless messages go somewhere the judge can see. That somewhere must provide two things together: a record of what was made available (publication), and deadlines against which the record is read (a clock). We call the combination a *public clock*. Bitcoin is one, at the cost of a transaction per message; the venue is one off-chain.

A single Bitcoin block provides three things at once: a clock, a publication, and uniqueness (one chain, one spend per output). Because they arrive together they are easily conflated. Off-chain they separate:

- **A public clock** is what a two-party game needs. The venue’s per-move record must be *decided* (the flag majority), but no reader other than the two players ever consults it, so nothing is gained by one player being shown a different record than the other: the players’ own one-time keys already forbid them from signing two versions of a move.
- **A seal** is what a *shared fact* needs: a fact that many contracts read, such as the owner of a name or the set of bids in an auction. If the venue could show one record to one contract and a different record to another, each would settle on a different world.

6.2. Shared facts, and what defends them

A true single-use seal (a Bitcoin output) reaches a contract only through a spend in the contract’s own inputs; that is how Lightning’s funding output works. A fact carried as *data*, whether a venue attestation, a Merkle root or a Bitcoin `OP_RETURN`, can only be exhibited to a script, and a script cannot tell which of two conflicting exhibits is canonical without verifying the chain it came from, which is impractical. A seal on Bitcoin (anchoring the venue’s record in a single-use output chain) therefore gives uniqueness to *clients* running nodes, but not to contracts.

For contracts, uniqueness of a shared fact is economic: two attestations for one fact are an unambiguous, attributable equivocation by the venue, whichever of them was anchored, and the condition of Section 3 must hold with G the value extractable from *all* contracts relying on the fact. For a single two-party game G is one pot, though a majority acting against many games at once faces the same aggregate (Section 3). For a shared fact it is an aggregate, measured against the franchise of the members who equivocate. We take this as the limitation of the system for shared facts: they are secure up to a value set by the venue’s fee revenue, bounded, and best suited to facts whose total reliance is bounded and known when they are published. An auction is such a fact; a long-lived registry is not.

7. Storage contracts: Filecoin without an altcoin

We return to the example of the introduction.

Set-up. The customer erasure-codes and encrypts its data, splits it into chunks of a few hundred bytes, and computes a Merkle root R over them. A contract in the customer’s channel with the provider fixes R , the number of chunks, an epoch length, a fee per epoch and the provider’s bond B . The data is handed over off-chain.

Each epoch is two moves. The customer publishes a challenge, an index i , on the venue. The provider must publish chunk c_i with its Merkle path to R by the next due time. The fee for the epoch is paid in the channel as usual; the bond stays locked.

Disputes are the game’s disputes. A missing response is an absence claim, refuted only by a venue attestation of a timely response. A wrong response is a disprove leaf that checks a Merkle path (SHA-256, native in Script; the chunk must fit a 520-byte stack element). A false claim by the customer is answered by the attested response. The contract graph is the chess graph with a different disprove family: the “move” is a challenge or a response, and the “rule” is a path check.

What it proves. A provider that has discarded a fraction f of the chunks survives each uniformly chosen challenge with probability $1 - f$. Because the data was erasure-coded, the customer needs only a sufficient fraction, so a provider passing most challenges holds a recoverable copy: this is a proof of retrievability in the sense of Juels–Kaliski and Shacham–Waters [18, 19]. It proves retrievability, not that the provider itself stores the data; for the customer that is usually what matters. Responses are public on the venue, hence the encryption. The contract’s depth limit bounds its term; longer terms are renewed by cooperative refresh, and the default at the horizon must favour the customer.

What is new. Paying for proofs of retrievability over Bitcoin has been proposed before: contingent service payments on Bitcoin for exactly this [20], recurring variants [21], periodic challenge-and-pay over a payment channel [22], and per-download payments with a bond and fraud proof over Lightning [33]. All are *pay on success*: if the provider does not answer, the customer does not pay, and the only penalty for losing the data is lost future income. No two-party construction can do better, because silence cannot be adjudicated between two parties. With a public clock “no timely response in epoch k ” is a fact, so the provider’s *bond* is forfeited for absence, not merely its payment withheld, and the whole schedule runs off-chain.

Sia and Filecoin. The dedicated storage networks do punish absence, and they can because each has its own blockchain, which gives them the public clock. A Sia host posts collateral in a file contract and must submit a storage proof for a randomly chosen segment within a proof window at the contract’s end; if it misses the window its collateral and earned revenue are burned, and the renter recovers only its unspent payment [29]. A Filecoin storage provider posts pledge and deal collateral and must answer a Proof-of-Spacetime challenge for every sector every day; a missed proof puts the sector in fault, with a daily fee, and persistent faults terminate it and slash the collateral, which is largely burned [30]. Both burn rather than compensate. In

Filecoin the reason is a third party: stored data earns storage power and block rewards, so a client and provider who could profit from penalties together would be taking from the network. A bilateral contract has no third party to defraud, and paying the customer from the bond is sound. What it does create, as Sia’s design anticipates, is the customer’s interest in the provider failing, by disrupting its connectivity or by colluding with the venue to suppress its responses; the second is the venue-majority attack of Sections 11 and 12, and the first is answered by the provider’s freedom to submit to any member, and to publish on Bitcoin in distress. And in both, the penalty deters but does not recover data: durability comes from erasure-coding across many independent providers (Sia’s renter software spreads each file across dozens of hosts and repairs as they fail), with frequent challenges to detect a loss while repair is still possible. The same holds here: the contract’s role is deterrence and early detection, one contract per provider. What we add is the absence-adjudicating clock without a blockchain or a token of its own: the venue provides it, and settlement stays in bitcoin, in a channel.

8. Hidden information: blackjack

A public-information game such as chess demonstrates the mechanism, but it is a thin market for money. The games people play for money hide information, and whether LN-GAP can host them is the more telling test. Blackjack is the simplest casino game that needs everything the venue provides, and it needs little else. The proof of concept [23] implements it: one hand per contract, played between a player and the house in a real channel, with a browser front end.

The game. A player plays against a dealer (the house). Cards count at face value from 2 to 10, ten for a jack, queen or king, and an ace counts one or eleven, whichever helps. The player receives two cards face up, the dealer one face up and one face down (the *hole* card). The player then repeatedly chooses to *hit* (take a card) or *stand*; going over 21 loses at once. When the player stands, the dealer reveals the hole card and draws by a fixed rule: take cards until the total is at least 17. Whoever is closer to 21 without going over wins; equal totals are a tie. We omit the refinements of casino play (3:2 for a natural 21, doubling, splitting, insurance).

Why it fits. The dealer makes no decisions: its play is fixed by the rule. So the hidden information is not anyone’s secret strategy, only the cards not yet dealt, and the protocol must guarantee three things: each card is random and neither side can bias it; nobody learns a card before it is dealt, the house included; and the rules, the dealer’s among them, are followed. The first two are commit-reveal randomness; the third is a disprove family of small arithmetic. What commit-reveal cannot do on its own is stop a party who sees that a card will go against it from refusing to reveal its share. That is exactly what the venue does: a reveal missing at its due time is a stall, and a stall loses the pot and, with dispute deposits, pays for the dispute.

Randomness. At open, for each card position $k = 1..K$ ($K \approx 16$ suffices for a hand) the player commits to a share a_k and the house to a share b_k , each in $0..12$; card k is $(a_k + b_k) \bmod 13$, a rank from ace to king. One honest party’s share is uniform and fixed before the other’s is known, so the card is uniform, and neither party knows it until both shares are revealed. Checking a share against its commitment in Script is the one new component, since the obvious commitment, a hash of the value and a salt, cannot be opened without concatenation. Instead the value is encoded in the *length* of a random string, as in the Bitcoin lottery of Andrychowicz et al. [31]:

commit $C = \text{SHA256}(s)$ with $|s| = 32 + v$ bytes; to open, reveal s , and the script checks the hash and reads v with `OP_SIZE`. The string is random and the hash hides its length; a second opening of a different length would be a SHA256 collision (the 80-bit collision bound of HASH160 is too low for a committer with time to grind). The dealing check, which opens both shares, bounds each to 0..12 and computes the card, is a 137-byte leaf; on regtest a correct spend is 183 vB, and a wrong preimage, a string one byte longer, an out-of-range share and a wrong claimed card are each rejected.

Where the strings travel. A share string cannot pass through the parked heads: the hash and `OP_SIZE` act on one stack element, and rejoining a string from parked nibbles would need `OP_CAT`. So the heads carry only the cards (one rank nibble each), and the strings ride in the entry’s *body*, beside the head and the mover’s signature. A disprove leaf that needs a share takes the string as its own witness element, checks it against the commitment pinned in the leaf at open, and reads the value with `OP_SIZE`. The claimant has the strings because entry bodies are public, and they are public because the members check them: for a blackjack contract, the registered check that already requires the mover’s signature also requires each entry to carry, for every card position whose share it reveals, a string that opens the mover’s commitment to a value in 0..12. An entry without them is not a move: honest members neither seal it nor count it at its due time. Availability is the venue’s; correctness stays with the leaves.

Play. The game is a sequence of venue entries, the player moving at odd depths and the house at even ones, each signed by its mover exactly as chess moves are. For every card the player reveals its share first and the house second. The player’s first entry reveals its shares of its two cards and the dealer’s up-card, and the house’s reply reveals its own, dealing them. Then the player either hits, revealing one share, which the house answers with the card, or stands, revealing its shares for the hole card and every possible dealer draw, which the house answers with the whole dealer phase in one entry: its play is forced, so nothing is lost by making it at once. Revealing second, the house learns each card first and could refuse to reveal one that goes against it; that refusal is a stall, and loses the pot. Every entry carries the claimed new state (the cards so far, the counters, the phase and the status), and a hand ends with a status that the channel settles.

Disputes. The disprove family covers: a share string that does not open to a value in 0..12; a card that is not the sum of its two shares modulo 13; an action not allowed in the phase, or the wrong next phase; counters not moved as the action requires; a dealt card changed; a wrong status (a bust, the showdown, a push); the dealer stopping below 17 with cards left, or drawing at 17 or above; and malformed fields. Each is a comparison or a few additions, of the size of tic-tac-toe’s leaves. Timeliness is the venue’s, as in any game. One detail differs from chess. There, a refuted depth’s result always favours its mover, so the mover’s checked split is gated by the mover’s outcome code. In blackjack the house can park a hand it lost, and could then lock the pot by never revealing its code; so blackjack’s checked split proves the result in the leaf without the code, and either party can broadcast it after $\delta + \delta'$.

In the channel. A hand is a contract of its own: adding it is a channel update carrying both stakes and both dispute deposits, and the winner proposes the update that removes it at the result, which the loser’s client accepts only if it computed the same result. A hand the loser

will not settle goes on chain by a force-close, and is disputed off the confirmed commitment's graph. On regtest, three honest hands settled in six channel updates with no chain transaction; a hand with a wrongly computed card was force-closed (commitment 180 vB), claimed (189 vB), refuted (24,282 vB) and disproved (4,916 vB). Chess runs in the same channel layer: a mate is settled by the winner, a resignation or an agreed draw by update, and an illegal move goes on chain the same way.

Simplifications. Cards are drawn from an infinite deck (each rank independently, ten-value cards four times as likely), which is close to the six- or eight-deck shoes casinos use and avoids shuffling in Script altogether; dealing without replacement would need a mental-poker shuffle, which is much heavier. Payouts are one to one, and a push refunds both stakes; the dealer stands on every 17. Sixteen card positions bound a hand. A session of many hands is a sequence of contracts in one channel, each added and settled in turn.

Why it matters. “Provably fair” online casinos let a player verify, after the fact, that the outcomes were honestly random, while the casino holds the player's money. Here the house never holds the player's funds, the cards are unbiassable, and the rules, the dealer's included, are enforced by the contract: provably trustless rather than provably fair. The house keeps its statistical edge, which comes from the dealer acting last, and that is legitimate. Babilonia [32] makes the same move for a one-shot bet, with a different emphasis: a single wager, resolved by an adaptor signature whose completion reveals the outcome, with zero-knowledge proofs that the dealer's hidden choice is well formed, and indistinguishable on chain from an ordinary payment. A one-shot bet needs no per-step clock, since Bitcoin's own timeout resolves its only step; blackjack has a decision at every card, and so needs the venue. The two combine: where one side genuinely holds a secret, as in poker, Babilonia-style reveals by adaptor completion could replace the length commitments.

9. A sealed-bid auction

An auction is the simplest useful contract that is not two-party. A seller auctions a lot; each bidder, and the seller, has a channel with a hub H ; every party's contract is a two-party contract with H , and all of them settle on one shared fact: the set of bids published on time.

Phases. Bidders publish commitments $H(\textit{bid} \parallel \textit{salt} \parallel \textit{key})$ by a commit time T_c and reveal them by a reveal time T_r . A bidder locks $L_i \geq \textit{bid}_i$ in its contract, over-locking by a random amount so that H learns nothing before the reveals. An unrevealed commitment forfeits a deposit π . On the keyed venue each bidder's commitment and reveal are entries under their own keys, one table per (auction, bidder, phase), so each has exactly one legitimate content and per-bidder timeliness comes from the flags as for a game; this needs the bidders known at registration. What the venue does not provide is the *set*: that these are all the bids, and that every reading contract sees the same ones.

The outcome is a pointer claim. Settlement is claimed as “winner w at price p , where p is bid k ”, not computed. The claim is refuted by exhibiting a timely revealed bid above w 's, a timely revealed bid other than w 's above p , or by showing that bid k is not a timely reveal. Each refutation is one inclusion path and one comparison; the number of bidders never enters a leaf.

Completeness comes from the same leaves: an omitted bid is exhibited with its timeliness, and the seller’s contract can exhibit *any* omitted bid, so H cannot suppress the high bid in favour of a shill.

The legs. As with a routed payment, the hub’s contracts net to zero: in the seller’s contract the lot goes to H against p , and in the winner’s the lot goes to the winner against p ; losers are refunded. The legs are atomic because they settle on the same fact.

Delivering the lot. If the lot is a Bitcoin-native asset (RGB or Taproot Assets), it is a Bitcoin output: the seller commits it into the contract output at open and delivery is a branch of the pre-signed graph, so the seller cannot withhold it later. Each graph transaction the asset passes through carries its asset transition, built in advance using outputs of the same transaction as seals; the asset’s history is published on the venue at set-up so the winner can validate it. If the lot lives on another chain, the auction sells the right to be the counterparty of an atomic swap at price p , and the seller’s option to walk away can only be priced, by a deposit forfeited on timeout.

Its security. The bid set is a shared fact. An equivocating venue could show the seller’s contract a high price and the winner’s a low one, at the hub’s expense. That is the limitation of Section 6, and here it is benign: the value at stake is bounded by the auction and known when its phases are registered, so the fees for its attestations can be sized to it.

What is still to build. Per-bidder registration, the pointer-claim graph, and settlement at a variable price (either a grid of pre-signed splits, one per price bucket, or payouts decomposed into binary-weighted outputs).

10. Arbitrary programs: disputing a zero-knowledge verifier

Every game so far has a hand-written disprove family: the rules are small enough that each way of breaking them fits a leaf. An arbitrary program has no such family, and its execution is far too long for any script. BitVMX [14] gives the standard answer for Bitcoin. The program runs on a RISC-V CPU whose execution trace is a hash chain, $h_i = \text{BLAKE3}(h_{i-1} \parallel w_i)$ truncated to 20 bytes, where w_i is step i ’s 13-byte write record (address, value, next program counter, micro-step). A prover claims that the program halts with success at step N with final hash h_N . A challenger who disagrees runs an n -ary search with the prover: in each round the prover posts the hashes at $n - 1$ evenly spaced steps of the disputed interval, and the challenger picks the sub-interval where they first diverge from its own. After about $\log_n N$ rounds the dispute is one step, whose predecessor hash both parties accept, and a single-step check decides it. In BitVMX every move of the search is a Bitcoin transaction. Here the moves are the game’s moves on the venue, like any other: a stalled round is an absence claim, and only the last step reaches the chain.

The last step is proved, not disproved. BitVMX’s step scripts are written for the prover: a chain of assertions (the opcode is of this instruction class, the reads come from the registers the opcode names, the memory witness matches) followed by an equality between the committed write and the computed one. Inverting only the final equality gives a disprove leaf that is sound

but not complete: an assertion that fails aborts the spend, and Script has no conditional way to succeed after one, so a read record inconsistent with its opcode fires no leaf. At the final step we therefore use BitVMX’s orientation. The prover’s refutation parks the disputed step’s record, and the refuted output offers, in windows that each belong to one side, the claimant’s disproves after δ ; the prover a **prove** leaf per instruction class after $\delta + \delta'$; and the claimant a timeout after $\delta + 2\delta'$, so a proof cannot pre-empt a disprove. A **prove** leaf is BitVMX’s script for the class unchanged, followed by the step’s hash checked with BitVMX’s BLAKE3, and preceded by a guard that the opcode belongs to the leaf’s class: two of BitVMX’s scripts never check the opcode, so without the guard any step could be “proved” as a no-op. The proof covers execution, not the truth of the reads, so BitVMX’s other verifier challenges become the claimant’s disproves on the same output: the program counter read is the previous step’s written one; the opcode is the program’s instruction at that address; the halt; reads of initialised and uninitialised memory; and reads of the input, which the prover signs word by word under one-time keys when it makes its claim. A read that claims to return the last value written to its address is checked as BitVMX checks it, by a second search for that write, played as a second phase of the same game, with BitVMX’s two challenges at its end.

The search as a game. Each round of the search is a move: the prover posts the hash at the midpoint of the disputed interval, and the verifier chooses a half (a binary search: BitVMX’s n -ary search with $n = 2$, its decision code unchanged). The search’s state (the two agreed endpoint hashes, the interval’s first step and a digest of the claim) is 64 bytes, more than a head holds; so, as for blackjack’s share strings, the entry’s body carries the state and its head the state’s 20-byte BLAKE3 digest. A leaf that needs the state takes it as witness and checks the digest in Script, and the members’ registered check requires every body to open its head’s digests, so the state is always available. A wrong transition fires a disprove like an illegal chess move, a stalled round is an absence claim, and only the last step reaches the chain.

What the proof of concept does. The program is BitVMX’s Groth16 verifier, a RISC-V build of a pairing-based verifier, checking a RISC0 [12] proof that a STARK receipt of a guest program is valid. On a genuine proof it halts with success after 478,727,216 steps. Challenged, BitVMX’s search code takes 29 rounds (some nine minutes off chain), which is a contract of 119 depths: 2,058 pre-signed transactions, with 1,919 disprove leaves and 62 **prove** leaves at the final depth of the first phase. All 59 moves of that phase were sealed by the venue, and on regtest the dispute was the claim (220 vB), the refutation (24.3 kB, as re-measured on a smaller program: its size depends on the heads, not the program) and the proof of the halting **ecall** (58.5 kB). The on-chain cost does not depend on the program’s length. The leaves are large (a **prove** leaf is about 228 KB of script, at a peak stack of 948 of the 1,000 allowed), but they depend only on the contract’s keys, so they are built once per contract and channel updates reuse them. The read challenge’s second phase runs end to end on regtest on a smaller program, through the fault cases of BitVMX’s own tests, but not yet on the Groth16 verifier. Two caveats: the verifier program’s public constants are set to the RISC0 version of the available proof; and the second phase’s first round is posted afresh rather than reused from the first, so a prover that posts a different hash there is not caught as such (BitVMX’s protocol does not give it the chance).

10.1. Claims that carry their own proof

A zero-knowledge verifier changes what a first move can be: not “I am owed X ” but “I am owed X , and here is a proof”. This bears on shared facts (Section 6.2), whose difficulty was that their truth could only be attested.

If a claim’s truth is a function of data fixed when the contract opens (the program or verifying key, the keys of any issuer, a genesis commitment) and of the prover’s own witness, no fact is shared between contracts at run time. The venue attests only when each contract’s moves were published, and the aggregate gain that bounds a shared fact’s security does not arise; the per-contract majority on timing remains, as for any game.

A bridge is not wholly of that kind. An entitlement to withdraw is a fact about a history that many parties extend after the contract opens, and setup can fix only the rules for recognising that history. Two questions remain. The first is which history is canonical, and a bridge has two histories to settle. The other system’s own order, its transfers and withdrawals, is typically fixed by a sequencer or a committee signing its states: their keys are one more setup fact, and the trust moves to them, with the same answers as for the venue (a bond, and the data published where others can see it). The deposits, though, are Bitcoin transactions, and whether they happened is not the sequencer’s to decide: one crediting a deposit that exists only on a fork would mint entitlements from nothing. That history is decided by proof of work, which is self-certifying: accumulated work is evidence, not testimony, like publication. The proven statement includes the deposit’s inclusion under a chain of headers, and a challenger may answer with a heavier chain, as a move of the game, so the question is settled by the game and the venue’s clock rather than by the venue vouching for a fact. This is how BitVM2 bridges treat it [13], and why ESSPI’s authors note that an inclusion proof must admit counter-proofs within a bounded time [16]. The second question is uniqueness: an entitlement proved in two contracts would be paid twice, which is the shared-fact failure again. It is answered inside the proven statement, by binding the entitlement to the contract that pays it (the contract’s identifier a public input of the proof). A stale history is not a separate danger if a withdrawal is a permanent burn on the other system, since every canonical history containing the burn will do.

Zero-knowledge claims therefore do not abolish shared facts; they convert them into setup facts, self-certifying evidence and bindings inside the proven statement, none of which the venue has to attest. A shared fact is a problem for this design only when its truth can merely be attested; when it can be proved, it is one more move of the game. The practical costs remain: the claim carries the proof, 256 bytes and its public inputs, which, like the search’s state, must ride in the entry’s body with its digest in the head; the other system’s data must be available to the claimant and to any challenger; and a counter-proof must be published before its deadline, the same assumption on the venue’s timing and on Bitcoin’s liveness as everywhere else.

10.2. Channels instead of bridges

A bridge moves bitcoin into a system that can express richer programs; this design moves the programs to the bitcoin. Most of the globality of a bridge follows from the first choice. Its deposits are pooled in jointly controlled outputs, and the pool’s contract cannot see Bitcoin, so every withdrawal must convince it, through a proof, that the deposits and the other system’s state lie on the canonical chain; the setup grows with the operators and the deposits, and the pool nets the risk of everyone who used it.

In a channel the funds never leave Bitcoin. A deposit is a funding output that each party checks with its own node, as in Lightning, so no script is ever shown that a deposit happened; each user's claim is backed by the collateral in its own channel, and a unilateral close puts it on chain with the contract attached. Nothing is pegged, so nothing is pegged out. What many parties share is reached through hubs: a statement about the hub's own accounting is a bilateral claim against the hub, provable in the user's contract (Section 10.1), and a fact of another channel, such as a payment made through the hub, crosses by Lightning's own device, a secret whose revelation in one channel is evidence in the next. Every fact a contract needs is then bilateral, fixed at setup, self-certifying, bound inside a proven statement, or a revealed secret, and the venue attests the content of none of them.

The price is locality. The venue's majority assumption holds per contract; liquidity is locked per channel, without the netting that makes a pool capital-efficient; and a contract can resolve only local state, so an application that needs a total order across many parties (price-time priority, first-come registration, an auction among many bidders) places that order with a hub, whose misbehaviour is then bounded by each channel's collateral rather than threatening a pool. In return there is far less pooled risk, and far less computation and on-chain weight in a dispute: nothing outside the channel is ever proved to Script, and a dispute stays within a constant number of transactions.

11. Security

Table 2 lists what can go wrong and what prevents it. The assumptions fall into five classes:

- **Cryptographic:** holds unconditionally given the primitives (one-time signatures, Schnorr, SHA-256) and Script.
- **Lightning's:** a party watches the chain and responds within its windows, and its transactions confirm within them (Bitcoin's liveness, below).
- **1 of n :** at least one member is honest and live before the relevant due time.
- **Majority:** fewer than t members collude.
- **Economic:** misbehaviour is attributable and costs more than it gains; bounded by the condition of Section 3.

threat	defended by	assumption
publishing a revoked channel state	revocation leaf	Lightning’s
an illegal move	disprove family, from the refutation’s witness	cryptographic
answering a claim with an entry the mover did not sign	authorship check in refute	cryptographic
a player signing two moves at one depth	equiv leaf	cryptographic
a stall	absence claim and timeout	Lightning’s
a false absence claim	refutation with the venue’s attestation	1 of n (some member sealed the move)
an absence claim at a depth the game never reached	counter	cryptographic
censorship of a mover (no member seals)	nothing in Script; fee income, member independence	1 of n
a late seal colluding with the mover	not_timely : t flags	majority
false flags against a timely move	none in Script; flaggers named; provable only with an audit layer (§12)	majority; economic
a committee majority lying about time	none in Script; publicly judgeable with an audit layer (§12)	majority; economic
an unsigned seal	provable from the seal; ejection	economic
two different signed moves at one depth	equiv leaf (the mover’s fault)	cryptographic
fee lock completed without sealing	t itself if the member sealed nothing for the move, else the claim’s provenance; ejection; the payer re-routes in time	economic
the venue halting	none: see below	—
$n - t + 1$ members withdrawing together (compelled, or offline)	none: sealing continues but no move can be shown late, so stalls go unpunished	committee composition
a shared fact shown differently to different contracts	attribution; ejection; optional burn	economic (G : aggregate)
refusing to refresh at the horizon	the horizon’s default outcome	design

Table 2: Threats and their defences.

Where the cryptography ends. Everything about a game’s *content* is cryptographic: a move is legal or a disprove leaf fires, a player signs once per depth or loses, a revoked state is swept. Everything about *time* rests on the venue: that a move was published is a 1-of- n fact, that it was published *on time* is a majority fact. This split is the design’s central trade. Content is expensive to adjudicate and made cheap by the disprove family; time is impossible to adjudicate between two parties and made possible by the venue.

A timekeeper, not a custodian. A first reading often takes the design for a federated sidechain with extra steps: if a majority decides the truth, why not let it decide everything?

The answer is in what the majority decides, and what it holds.

- *What it decides.* A federation's keys sign every withdrawal, so its majority can move any funds anywhere. The venue's committee holds no key to any funds and decides one bit per move. It cannot forge a move (the player signs it), change a rule (Script enforces it), or pay anyone.
- *Whom it needs.* A federation majority can steal alone. A venue majority gains only by helping one party of a contract against the other, so it needs that party as an accomplice, and a dispute that reaches the chain and turns on timing.
- *How much is exposed.* A federation's majority reaches every coin in its custody at once. A committee's reaches the pot of a contract it serves, in a live dispute, and committees drawn per contract (Section 12.2) make a broad attack need a majority of many committees.
- *Freezing.* Theft is not the only risk, nor the most likely: a custodian can refuse to let funds leave, and a federation of named legal entities can be compelled to, and can do it easily, for instance by admitting withdrawals only to approved addresses. Here there is nothing to freeze. Every user's funds sit in its own channel, a unilateral close is always available, and if the venue falls silent the contracts still resolve on Bitcoin (below).
- *Who the members are.* A federation's members must be known, since users trust them by name and their legal exposure is part of the security, which is also what makes them compellable. The venue's members need no legal identity: each is a bonded output and a key. Whether they are pseudonymous members of an open roster or identified operators with reputations at stake is a choice for the roster and the contracting parties, and committees can mix the two. The two kinds of guarantee pull different ways: open, pseudonymous entry can only help the 1-of- n guarantees, since one honest member suffices and Sybils cost nothing there, while the majority guarantees are exactly what Sybils attack, and members with reputations at stake add a cost beyond the bond. A committee mixing identified operators with members of an open roster, across jurisdictions, is the configuration this suggests; the design does not require it. Either way, compelling a member achieves little, since it holds nothing; the most it can be made to do is misstate the time of a move in a contract it serves, under its own key.

The machinery of the preceding sections exists to shrink what the majority decides. Script judges content, one-time signatures bind each move to its author, publication needs one member, and the majority is left with a single bit. Without that machinery the majority would decide outcomes, and the design would be a federation. In a federated sidechain the majority is judge, jury and bank; here it is a timekeeper, which can be bribed to misstate the time of one move in one contract, and only in league with a party to that contract.

Censorship. A mover whose entry no member will seal loses by absence. This is theft-shaped, not merely a delay, and nothing in Script prevents it. It requires every member to refuse until the move is due, and so is defended by the independence of the members (operators, jurisdictions, hosts) and by the fees each forgoes; bonds, which punish only self-contradiction, do not reach it. Publishing the entry on Bitcoin makes such a refusal publicly visible (Section 12).

What is 1 of n , and what is not. Publication is 1 of n : one live, honest member seals a move. Timeliness is not. Flags count only when t members reveal them, so `not_timely` needs t members live and honest at the due time, and any $n - t + 1$ members acting together, by collusion or by compulsion, can switch it off. Consider a committee whose majority are companies in one jurisdiction that, partway through a contract, are obliged by law to stop serving its parties. They cannot freeze the funds, which they never held, and must not side with either party. The other members still seal, so the game can go on; but no move can now be shown to be late, so an absence claim is no longer safe against a deliberate staller, who answers it with a late move sealed by a remaining member, leaving the claimant no flags to show. (The claim still wins against a party that has truly gone.) Stalling becomes a free option for whoever is losing, and the contract falls back on what needs no flags: a cooperative settlement, the deadline’s `settle`, or a fresh contract under another committee (the members are fixed at open, since the graph embeds their points; a protocol to move a live contract to a new committee is future work). That is far better than a custodian’s freeze, but it is a degradation, and it shows what a committee’s composition must guard against: not only a colluding group of t , but any group of $n - t + 1$ that can be compelled together, such as one jurisdiction, host or operator.

Bitcoin’s liveness. Every branch of the dispute graph is a race against a timelock: a refutation must confirm before the claimant’s timeout, a disprove before the mover’s split. The contract therefore assumes, as Lightning does for its HTLCs and BitVMX for its rounds, that a transaction can be confirmed within its window. The graph’s pre-signed transactions have one input each and pay a fee fixed at signing, out of that input; this leaves the replacement-cycling attack [15] no second input to evict them through, but a fee spike can hold one back past its window. Fee-bumping (an anchor output for CPFP) would answer the spike and readmit the attack, with Lightning’s mitigations; the windows δ and δ' are what buys time for either.

The majority. Flags are the only place the design needs a threshold, and it is there for *order*, not for stake: two statements by two members (“move d was not published”, “here is move d ”) cannot be ordered by any third fact, so canonical timing needs agreement among the members, or an external clock. A colluding majority can kill an honest mover’s refutation with false flags; the flags are signatures, so the colluders are named, and the residual falls on a party who knows whom to pursue. Naming is not proof, however: whether a move was published in time cannot be shown to a third party. Section 12 describes how to make it so.

Venue failure. If the venue stops altogether, nothing can be published, and an absence claim against whoever is on move succeeds. In the present design a dead venue therefore decides games against the player on move. Requiring the claimant to show the flags (the positive statement “move d was not published”) *in the claim itself* would turn this into a liveness failure (no flags, no claim; the contract settles by its default at the deadline), at the price of letting a colluding majority protect a staller by withholding flags. The choice is between the two residuals, and it is open.

Economic security, and its limit. The bond and the fees defend what is attributable. For one two-party contract the gain from venue misbehaviour is at most its pot, but a colluding majority can act against every contract it serves at once, and a hub is a colluding player in many of them; so the members’ fee franchise must exceed the value in flight across the venue,

not the largest single pot. For shared facts the gain is the sum over all contracts relying on a fact, and the same franchise must cover it. We state this as the limitation of the system rather than engineer around it: the only ways to remove it are verifying the venue's chain in Script (SPV, impractical) or turning the fact into Bitcoin outputs that each contract spends, which works only for facts of a few bits fixed in structure long in advance. Two further courses accept the limit but shrink it: committees drawn per contract, so that a targeted attack reaches one pot and a broad one needs a share of a large roster; and accountability rather than enforcement, making misbehaviour publicly verifiable (Section 12).

12. Weakening the majority assumption

This section describes extensions that are not yet implemented. They address the one threat of Section 11 that the design leaves to economics alone: a colluding majority of the venue. There are two levers: *whose* majority it is (a committee drawn per contract from a bonded roster), and whether its lie *can be shown* (publication on Bitcoin, or an audit layer). The threshold itself stays a bare majority (Section 12.3 says why).

12.1. The threshold adversary

A majority of the members can take the pot of any contract, in either direction. By *false flags*, t members declare an honest mover's timely move unpublished and `not_timely` kills its refutation. By a *late seal with suppressed flags*, one member seals a stalling mover's move after its due time while $n - t + 1$ members withhold their flags, so the late refutation stands. The majority threshold balances the two, each needing about half of the members. This resembles a majority attack on a proof-of-work chain, with three differences. The venue's attackers are identified (every flag and every seal carries a member's key: a pseudonymous identity, not a legal one), where a miner's work carries no identity at all. What they stake is a franchise of future fees, bought with a timelocked bond that cannot be rented for the attack, where hash power can. And the attack can target a single contract, silently, where a reorganisation is public.

What makes the attack hard to punish is not anonymity but the nature of the fact decided. Whether a move reached the members in time is exactly the kind of fact that no third party can check: the impossibility of Section 6, which the venue resolved for the two players, reappears among the members, and is resolved by vote. A timeliness lie by a majority is therefore attributable (we know who flagged) but not falsifiable (nobody can show the flag was false). Punishment by ejection needs an objective basis, and there is none.

How much that matters depends on the timescale. A timing lie is deniable only within what nobody can measure: the disagreement between clocks, which is seconds, and the vagaries of delivery, which are seconds to minutes. Against a move window of a minute, both are large, and "it was late" or "it never reached me" can be sustained. Against a window of hours, as for a daily storage proof or a withdrawal claim, they are not: a mover can send its entry to every member, repeatedly and by several routes, and post it where many independent parties see it, and a majority that then claims not to have seen it in time claims something no one will believe, though Script still cannot tell. (What counts here is publication, the entry placed before others, not a proof of its existence: a timestamp shows that the mover held the entry, not that anyone received it.) This is the reliance Bitcoin itself makes: flood-fill propagation is unreliable at the scale of seconds and dependable at the scale of hours, and nobody can stop a signal that has

hours to spread. The venue is a binding agent of the same kind, not very strong, but strong enough: it turns the imprecise public fact that something was widely published well before a deadline into one bit that Script can read. Its timing assumption is therefore weakest for fast contracts and weak only in principle for slow ones, and many of the valuable applications are slow.

12.2. Committees drawn from a roster

The first question is *whose* majority. Two players cannot both choose a majority of the members who judge them, and today neither chooses: the venue is one fixed set, and its majority is the same for every contract. It need not be. The venue is already per contract in everything but its membership: tables, proposer and flag points, due times and the registry are all keyed by the contract (Section 3), and nothing ties two contracts' members together. We propose that each contract's members, its *committee*, be drawn at random from a larger *roster* of bonded members. There is then no single venue but one per contract, each sampled from the same roster. A member's work falls rather than rises: it serves only the contracts it is drawn for, about n/R of them for committees of n from a roster of R . The committee's size is limited by Script (its flag points are counted in a leaf); the roster's is not.

The draw. Both players commit to random values before either knows the result. The seed hashes both values together with the hash of a Bitcoin block at a height fixed after the commitments, and a public procedure maps the seed and the roster to a committee. Without the block hash, whoever reveals last would see the committee first and could abort and retry until the draw suited it; with it, each attempt costs a block, and a player that keeps abandoning contracts before funding is conspicuous. Each player may first strike a small, fixed number of members it distrusts; a larger allowance would let a player strike honest members too. The committee is fixed at open, because the pre-signed graph embeds its points.

What it buys. An adversary now needs a share of the roster, not a majority of one set. If it controls a fraction f of the roster, the chance that a committee of a few dozen has a colluding majority is small while f is well below a half, and it falls exponentially with the committee's size. Collusion now takes one of two forms, and both cost more than before. A *broad* attack, taking many contracts at once, needs a share of the whole roster near a half: what a fixed venue's majority already posed, spread over a larger set. A *targeted* attack, bribing a particular committee after the draw, reaches only the contract that committee serves. The aggregate gain G of Section 3, which for a fixed venue is the value in flight across every contract (a hub being a colluding player in many of them), becomes for a targeted attack about one pot, and a hub that wants to steal from many of its contracts must corrupt as many different committees.

What it does not buy. Sampling defeats collusion arranged before the draw, not bribery after it: the committee is known at open and serves for the contract's life. It complements, and does not replace, the measures below, which make a lie by the chosen committee provable.

The bond is the ticket. Every guarantee above is a statement about the adversary's share of the roster, and that share means something only if it is costly to acquire. Membership of the roster is therefore a bond, and the bond does more work than in a fixed venue:

- *The roster lives on Bitcoin.* A member is a timelocked output with its owner’s key, announced by a signature under that key. The roster at a given height is the set of unexpired announced bonds, which anyone with a node can reconstruct. Entry is permissionless, as it is for JoinMarket’s makers, whose fidelity bonds [9] are exactly such announced timelocked outputs, and the draw is reproducible by anyone from the seed.
- *Weight, not headcount.* The draw weights members by the value locked and by the lock’s remaining term, so an adversary’s share of the roster is its share of locked bitcoin-time, not its number of identities. Splitting a bond into ten members buys nothing. JoinMarket goes further: it values a bond by the time value it sacrifices raised to a power above one, so that splitting loses weight and consolidating gains it. That trades Sybil resistance against concentration in a few large members, and for a roster whose purpose is a random, independent committee the linear weighting may be the better choice; we leave it open.
- *The term covers the contract.* A bond must outlast every contract its owner serves, including the dispute windows; a member whose lock expires before a contract’s last deadline is not eligible for that contract.
- *Sizing per contract.* A targeted bribe must pay a majority of the committee at least what each member stands to lose: its expected fees for the rest of its term. Those fees are not visible, but the time value of the member’s remaining lock is, and it bounds them from below (a member locks only if its fees repay it). The players can check that bound against their pot at open, and a contract with a larger pot can require a larger committee or a larger minimum bond. The condition of Section 3 becomes a per-contract parameter rather than one inequality for the whole venue.
- *Size and age.* A player has two levers it can check on chain at open: the size of each member’s bond, which bounds the member’s fee income from below, and its age, since a long-standing member may earn more than its lock and so has more to lose; weighting by age also makes an attacker pay in time as well as capital. Neither buys more than a franchise’s worth of defence: a player can reasonably expect protection for a pot up to about what a committee majority stands to lose, and no more.
- *Assembling versus using.* As in Section 3, the baseline bond buys a franchise that ejection takes away, and ejection needs an objective fault. A timeliness lie is attributable but not falsifiable (Section 12.1), so the bond makes a colluding majority costly to *assemble*; making it costly to *use* needs the record of the audit layer below.

12.3. The threshold: why a bare majority

A suggestion, not built: *two-sided votes*. Today a member’s silence counts in effect as “published”, so members protecting a late move need only stay quiet, indistinguishable from members that are offline. If each member instead held two one-bit keys per move, “published in time” and “not published in time”, and revealed one, a colluder would have to sign a false statement rather than hide behind silence, turning a deniable omission into an attributable lie; and a member revealing both would contradict itself provably.

Two-sided votes suggest a higher threshold: two thirds of “not published” pays the claimant, two thirds of “published” the mover, and anything between refunds both stakes, so that stealing

in either direction needs two thirds. We do not adopt it, because a refund is not a neutral outcome. The party most willing to bribe members is the one about to lose, and to it a refund is an escape: with the middle zone, a coalition of just over a third converts a loss into a refund, where today changing the outcome needs a majority. Other middle outcomes fare no better: burning both stakes invites grieving by the same third, a default to either side lowers that side's opponent's bar instead, and an appeal to a further committee only postpones the choice. With two outcomes and a decision that must always be reached, a two-thirds bar in both directions needs a third outcome, and every third outcome is an escape, a punishment or a postponement. The bare majority stays; the cost of an attack is raised instead by the other levers of this section.

12.4. Why not publish on Bitcoin?

The obvious public record is Bitcoin itself. A mover posts its entry in an `OP_RETURN`, confirmed with median-time-past before the move's due time, and members are required to watch for such postings. Publication is self-certifying (Section 3.3), so flagging a move that Bitcoin shows as published becomes a lie anyone with a node can see; if posting is mandatory, absence becomes checkable too.

Mandatory posting is not a solution. It puts every move on Bitcoin in the honest case, which is the per-move cost the venue exists to avoid. The cost is not quite that of an on-chain game, since a posting carries the entry and not the verification of the move, which for a game with weighty rules is most of the cost; but it is still a transaction per move, with confirmation times and fee spikes setting the pace of play.

Posting only in distress is more reasonable, as an option. With a long move window, say two hours, a mover that sees no seal within minutes of submitting, where it expects one within seconds, posts its entry then. This happens rarely, and the price is paid in long timeouts rather than fees. Its reach is limited in two ways. It answers a false flag, since the posting shows anyone with a node that the move was published in time; it does not answer the opposite lie, a majority that covers for a mover by sealing its late move or withholding its flags, because when posting is optional a missing posting proves nothing, and absence still has no public witness. And the evidence is for observers, not Script: the flags still decide the contract, and the posting makes the lie provable, to be punished outside it (the roster's bond, Section 12.2).

Posting on Bitcoin before a deadline is the right tool for a different job: one large publication per dispute, not one per move. ESSPI [16] builds it for the program input of BitVMX [14]: a pre-signed kick-off transaction has an output that the prover must spend, with a transaction carrying the input, before a relative timelock expires, or a pre-signed penalty takes the funds. Script is convinced of availability by the timelock, not by a proof of inclusion. Its authors observe that an inclusion proof, to be safe at high value, must admit counter-proofs within a bounded time, so it too rests on transactions not being censored for long, and the timelock asks for nothing more. The venue's contracts rest on the same assumption: every claim, refutation and timeout supposes that a transaction can be confirmed within its window.

Both modes show that a public record, not Script, is what makes the venue's honesty checkable. The question is where that record should live.

12.5. A third layer

The record need not live on Bitcoin, and it need not be verified in Script. Its only purpose is to let observers judge the venue after the fact, and observers can verify far more than a script can. The roster’s members are its natural participants, long-lived and bonded, but the roles differ: the roster is who may be drawn, the audit layer is the record against which the drawn are judged. This suggests three layers (Figure 4):

layer	role	verified by	constraint
Bitcoin	money; the dispute’s clock; rare anchoring of the audit layer	Script, everyone	slow, expensive
venue committee	per-contract attestations and flags, legible to Script	Script	small: tens to about a hundred members, one-bit statements
audit layer	the public record of what was published, and when	observers only	none from Script: large, dynamic

Every entry is published to the audit layer as it is made, cheaply and quickly; the audit layer commits its history to Bitcoin periodically (an `OP_RETURN` per hour, say, for all contracts at once), which fixes it against rewriting at an amortised cost. A committee majority that flags a move the audit layer shows as published, or lets stand a seal the audit layer shows as late, by more than a tolerance for clock skew and propagation, is then caught in either direction, without per-move Bitcoin cost. The pattern is familiar: a small committee that a constrained verifier can check, attesting to a larger system that full observers check, as Ethereum’s sync committees do for light clients [26]; and a public log whose timeliness rests on many independent witnesses and on anchoring, as in witness-cosigned transparency logs [27] and OpenTimestamps [6]. Script never verifies the audit layer, so the limit Script places on quorum size no longer limits the size of the trusted set; it limits only each contract’s committee.

The audit layer can be an existing blockchain chosen for cheap postings: the mover posts each entry there, and the chain’s own timestamps and finality become the record against which the committee is judged. What that chain must offer is availability of the posting, finality within the claim margin, and freedom from censorship within the move window, since a mover whose posting is excluded until its due time forfeits. It does not remove the committee: Script cannot read the other chain’s consensus, so someone must still attest, in keys that Script checks, what the chain shows. The contract remains denominated in bitcoin while the timeliness of its moves rests on another system’s trust model, much as a DLC’s payout rests on its oracle.

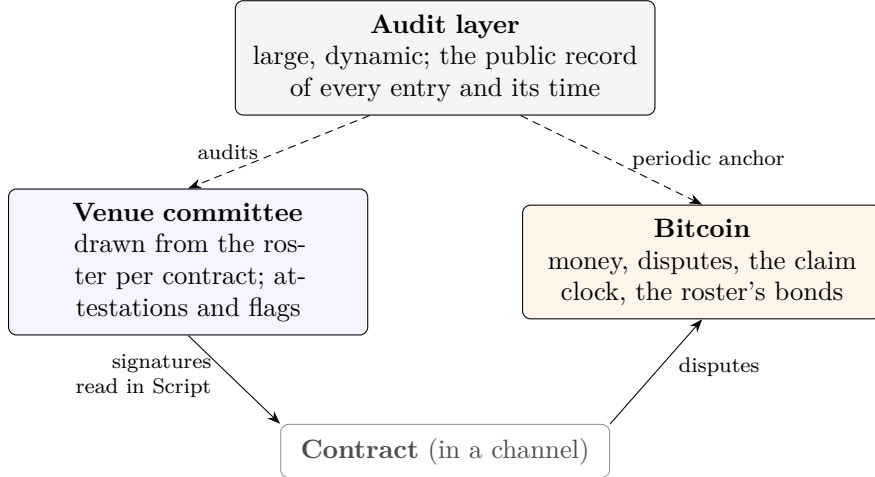


Figure 4: Three layers. Only the solid arrows are checked in Script; the audit layer makes the committee’s statements publicly judgeable after the fact.

12.6. What remains

A committee majority can still take a pot. With an audit layer it can do so only provably, at the cost of every colluder’s franchise; security remains economic at the top, like proof of work, but the attack is no longer deniable. One step further is possible: if committee members’ bonds are held under a key controlled by the roster (a large-threshold key over its membership), the roster could slash a provably colluding committee and compensate the victim, judging from the audit layer’s record. That moves the trust up a level, to the larger set, which is the point.

13. Related work

ForceMove [3] and state channels are the paradigm we modify; Lightning [1] the base we build on. Single-use seals and proof of publication are due to Todd [5], and OpenTimestamps [6], also his, deploys the related proof of existence (a timestamp, not a publication); RGB [7] builds client-side-validated assets on single-use seals, and its companion Prime design places commitments in a dedicated header chain anchored to Bitcoin. The bonded, Bitcoin-collateralised validator set with one-time signatures is Linus’s Coins and Stakechains [8]; what is new here is that the venue settles no value and carries no asset, and that its attestations are read in Bitcoin Script. BitVM [11] settles arbitrary computation on Bitcoin by bisection, and BitVMX [14] does so for a RISC-V CPU whose execution trace is a hash chain. There the search itself is on chain: a dispute is a graph of transactions pre-signed by both parties at setup, emulating covenants, in which each move of the search (the prover’s hashes, the challenger’s choice) is a transaction, the parties’ data are carried in Winternitz signatures that link the rounds, and each round is guarded by a relative timelock that a party who does not answer loses by. This is ForceMove played on Bitcoin with the rounds as moves. It needs no trust beyond Bitcoin’s eventual freedom from censorship, and a dispute costs a number of transactions, and of timelock waits, logarithmic in the length of the trace. We avoid on-chain bisection by giving each game a family of single-step disprove leaves, and where a program is too large for that, we play BitVMX’s search as a venue game, with only its final step on chain (Section 10); its CPU, emulator, search, step

scripts and Groth16 verifier program are taken from the BitVMX codebase. ESSPI [16], from the same authors, publishes large program inputs on Bitcoin before a timelocked deadline: the public clock of this paper, used once per dispute rather than once per move. StackSat-128 [17] is a Script-friendly hash designed for the proof-of-work alternative. The attestation primitive is that of Discreet Log Contracts [24]: an oracle publishes anticipation points in advance and reveals the scalar for the outcome that occurred, and a contract conditions on it; the venue is a federation of such oracles, attesting publication and timeliness rather than external events, and its attestations are read out in Script rather than consumed by adaptor signatures alone. The accountable venue of Section 12 follows the pattern of Ethereum’s sync committees [26] (a sampled committee attesting for a constrained verifier) and of witness-cosigned transparency logs [27]; committee sampling by joint randomness is in the spirit of Algorand’s sortition [28]. For storage, see Section 7; for hidden-information games and Babilonia, Section 8.

14. Conclusion

Lightning cannot adjudicate silence, and that single gap is what keeps games, service agreements and auctions off it. A venue that publishes each move as it is published, attests it with signatures a Bitcoin script can read, and flags by majority what it did not, closes the gap with $O(1)$ transactions per dispute and no change to Bitcoin. Content stays cryptographic; time becomes a 1-of- n and majority fact about a bonded, fee-paid set of members; shared facts are secured economically, up to a bound set by the venue’s revenue. A proof of concept is available [23]: the venue; the dispute graph for tic-tac-toe and chess, with their scenario suites on regtest; blackjack and chess played in a real channel, in the browser; and a dispute over a Groth16 verifier’s execution, searched on the venue with one step on chain. The remaining weakness, a colluding majority of the venue, can be made both harder to assemble and accountable rather than deniable: committees drawn per contract from a large bonded roster, and an audit layer, anchored to Bitcoin, against which their statements can be judged.

References

- [1] J. Poon and T. Dryja. The Bitcoin Lightning Network: Scalable off-chain instant payments, 2016. <https://lightning.network/lightning-network-paper.pdf>
- [2] A. Poelstra. Script state from Lamport signatures. Blockstream blog, 3 July 2024. <https://blog.blockstream.com/script-state-from-lamport-signatures/>
- [3] T. Close and A. Stewart. ForceMove: an n -party state channel protocol. Magmo, 2018.
- [4] H. Pagnia and F. C. Gärtner. On the impossibility of fair exchange without a trusted third party. Technical report TUD-BS-1999-02, Darmstadt University of Technology, 1999.
- [5] P. Todd. Preventing consensus fraud with commitments and single-use-seals, 2016. <https://peter todd.org/2016/commitments-and-single-use-seals>
- [6] OpenTimestamps. <https://opentimestamps.org>
- [7] RGB Protocol. <https://rgb.info>

- [8] R. Linus. Coins: a billion Bitcoin users, 2020; R. Linus and B. Tse, Stakechains, CoDecFin 2022.
- [9] C. Belcher. Design for improving JoinMarket’s resistance to sybil attacks using fidelity bonds, 2019. <https://gist.github.com/chris-belcher/18ea0e6acdb885a2bfbdee43dcd6b5af>; and JoinMarket, *Fidelity bonds* (documentation). <https://github.com/JoinMarket-Org/joinmarket-clientserver/blob/master/docs/fidelity-bonds.md>
- [10] J. Nick, A. Poelstra and G. Sanders. Liquid: a Bitcoin sidechain, 2020. <https://blockstream.com/assets/downloads/pdf/liquid-whitepaper.pdf>
- [11] R. Linus. BitVM: compute anything on Bitcoin, 2023.
- [12] RISC Zero. The RISC Zero zkVM and its Groth16 receipts. <https://dev.risczero.com>
- [13] R. Linus, L. Aumayr, A. Zamyatin, A. Pelosi, Z. Avarikioti and M. Maffei. BitVM2: bridging Bitcoin to second layers, 2024. https://bitvm.org/bitvm_bridge.pdf
- [14] S. D. Lerner, R. Amela, S. Mishra, M. Jonas and J. Álvarez Cid-Fuentes. BitVMX: a CPU for universal computation on Bitcoin, 2024. <https://arxiv.org/abs/2405.06842>
- [15] A. Riard. Replacement cycling attacks on the Lightning Network, 2023. <https://github.com/ariard/mempool-research/blob/2023-10-replacement-paper/replacement-cycling.pdf>
- [16] S. D. Lerner, M. Jonas and A. Futoransky. ESSPI: ECDSA/Schnorr signed program input for BitVMX, 2025. <https://arxiv.org/abs/2503.02772>
- [17] AbdelStark. StackSat-128, 2025. <https://github.com/starkware-bitcoin/stacksat128>
- [18] A. Juels and B. S. Kaliski. PORs: proofs of retrievability for large files. ACM CCS 2007.
- [19] H. Shacham and B. Waters. Compact proofs of retrievability. ASIACRYPT 2008.
- [20] M. Campanelli, R. Gennaro, S. Goldfeder and L. Nizzardo. Zero-knowledge contingent payments revisited: attacks and payments for services. ACM CCS 2017. <https://eprint.iacr.org/2017/566>
- [21] A. Abadi, S. J. Murdoch and T. Zacharias. Recurring contingent payment for proofs of retrievability, 2021. <https://eprint.iacr.org/2021/1145>
- [22] sCrypt. Pay for storage using proof of retrievability on Bitcoin. <https://scryptplatform.medium.com/pay-for-storage-using-proof-of-retrievability-on-bitcoin-d5f2df009e1>
- [23] LN-GAP proof-of-concept code. <https://github.com/AdamISZ/ln-gap>
- [24] T. Dryja. Discreet Log Contracts, 2017. <https://adiabat.github.io/dlc.pdf>
- [25] Ethereum consensus specifications, Phase 0: slashing and the correlation penalty (PROPORTIONAL_SLASHING_MULTIPLIER). <https://github.com/ethereum/consensus-specs>

- [26] Ethereum consensus specifications, Altair: the sync committee. <https://github.com/ethereum/consensus-specs>
- [27] B. Laurie, A. Langley and E. Kasper. Certificate Transparency. RFC 6962, 2013.
- [28] Y. Gilad, R. Hemo, S. Micali, G. Vlachos and N. Zeldovich. Algorand: scaling Byzantine agreements for cryptocurrencies. SOSP 2017.
- [29] Sia Foundation. About providing storage on Sia. <https://docs.sia.tech/provide-storage/about-hosting-on-sia>
- [30] Filecoin documentation: slashing. <https://docs.filecoin.io/storage-providers/filecoin-economics/slashing>; storage deal flow, https://spec.filecoin.io/systems/filecoin_markets/onchain_storage_market/storage_deal_flow/
- [31] M. Andrychowicz, S. Dziembowski, D. Malinowski and Ł. Mazurek. Secure multiparty computations on Bitcoin. IEEE Symposium on Security and Privacy, 2014. <https://eprint.iacr.org/2013/784>
- [32] waxwing. Babilonia: probabilistic coinjoin and covert betting. Version 2.0 (draft), September 2026. <https://github.com/AdamISZ/babilonia-paper>
- [33] R. Linus. BitStream: decentralized file hosting incentivised via Bitcoin payments, 2023. <https://robinlinus.com/bitstream.pdf>

A. The chess disprove family, in full

Section 4.4 gives the shape of a disprove_π leaf and names the predicates only by example. This appendix lists every predicate of the chess family, says what each computes and from which digits, and works through one disproof from the parked register file to the spend. The code is the `lngap-chess` crate (`certificate.rs` for the statements, `leaf.rs` for their Script) and the PoS glue in `lngap-pos` (`chess.rs`) [23].

A.1. What a disprove leaf sees

A confirmed refutation at depth d has parked, under the one-time key K_d^{ref} , the pair $\text{head}(d-1) \parallel \text{head}(d)$: 2×48 bytes, so 192 hexadecimal digits (nibbles, high nibble of each byte first). The disprove leaf re-verifies that Winternitz signature (`WOTS-VERIFY`), which leaves the 192 digits on the stack as the *register file*, and evaluates its predicate over them. At $d = 1$ there is no prior head on chain; the leaf pushes the constant initial head in its place, so the predicate always sees two heads.

A head is three parts:

bytes	digits (in a head)	content
0–3	0–7	word0: game id (2 bytes, big-endian), depth, mover
4–7	8–15	word1: the move, (promo $\ll 12$ to $\ll 6$ from) $\ll 16$
8–47	16–95	the state, 40 bytes (below)

In the register file the prior head occupies digits 0–95 and the new head 96–191, so the prior

state starts at digit 16 and the new state at digit 112. The state bytes are:

state byte	state digits	field
0–31	0–63	the board: one nibble per square, square s at digit s
32	64, 65	side to move (0 White, 1 Black)
33	66, 67	castling rights (4 bits; the high nibble must be 0)
34	68, 69	en passant square (64 = none)
35	70, 71	halfmove clock
36, 37	72–75	the move’s from and to squares
38	76, 77	promotion piece in the high nibble; low nibble 0
39	78, 79	depth

Squares are numbered $s = f + 8r$ with file f and rank r from 0, so a1 = 0, h1 = 7, a8 = 56. A square’s nibble is empty (0) or a piece kind k (1 pawn, 2 knight, 3 bishop, 4 rook, 5 queen, 6 king) plus 8 for Black. There is no move number in the state (the depth takes its place), which is why the chess certificate’s thirteenth kind, MOVENUMBER, is not part of the PoS family.

The common preamble. Script has no multiplication, division or bitwise operations, so every chess leaf opens with the same arithmetic, built from comparisons, additions and doublings:

- a byte from its two digits: $16h + \ell$ as h doubled four times, plus 1;
- rank and file of a square s : $r = \sum_{k=1}^7 [s \geq 8k]$ (seven comparisons), $f = s - 8r$ ($8r$ by three doublings);
- colour and kind of a nibble n : $c = [n \geq 8]$, $k = n - 8c$;
- a board read at a computed square: `<base> OP_SWAP OP_SUB OP_PICK`, the square having been range-checked to 0..63 (`OP_WITHIN OP_VERIFY`) whenever it comes from the witness.

From these the preamble names: the side to move S (from the *prior* state), $\Delta f = f(\text{to}) - f(\text{from})$, $\Delta r = r(\text{to}) - r(\text{from})$, and the kind K of the piece on from in the prior board, or 0 if that square does not hold a piece of side S . from, to and the promotion come from the *new* state’s bytes 36–38 (the mover’s own account of its move).

The general form. Every leaf of the family has the shape of §4.4:

$\langle \delta \rangle$	OP_CSV OP_DROP	$\langle P_{c(d)} \rangle$	OP_CHECKSIGVERIFY	the claimant, after the window
	WOTS-VERIFY(K_d^{ref})			the 192 parked digits
	PREAMBLE BODY(π)			one element: 1 iff the rule is
	broken			
	CLEANUP			drop the rest, leave that
	element			

and its witness, bottom to top, is: the exhibit (zero to two script numbers), the 195 (hash, digit) pairs of the Winternitz reveal (192 message digits and 3 checksum digits), copied from the refutation’s witness, and the claimant’s signature.

A.2. The predicates

Each chess predicate is the *negation* of one statement a legal move makes true; the leaf fires (leaves 1) exactly when the parked tuple falsifies it. “Prior” and “new” are the two parked states; S is the side to move in the prior.

leaf	fires iff	exhibit
wrong_slot	either head's word0 is not this game's constant for its slot: (game, $d - 1$, the claimant) and (game, d , the mover)	–
chess_malformed	the new head is not a well-formed encoding: from or to ≥ 64 ; the castling byte's high nibble or byte 38's low nibble non-zero; a promotion code of 1 or ≥ 6 ; the depth byte $\neq d$; or word1 is not the move rebuilt from bytes 36–38	–
chess_mover	from does not hold a piece of side S , or that piece cannot make the move $(\Delta f, \Delta r)$: a pawn one forward, two forward from its start rank, or one diagonally forward; a knight's L; a bishop's diagonal, a rook's line, a queen's either; a king's step or the castling shape ($\Delta r = 0$, $ \Delta f = 2$)	–
chess_destination	to holds a piece of side S ; or, for a pawn, to is occupied on a push, or neither an enemy piece nor the en passant square on a capture	–
chess_ray	the move is a line (not a knight's), $1 \leq j < j$ $\max(\Delta f , \Delta r)$, and the j -th square from from towards to is occupied in the prior. Covers sliders, the double push and the castling king's path	j
chess_promotion	a pawn reaching the last rank names no promotion piece, or any other move names one	–
chess_castling	a castling-shaped king move with: the king not on its home square, the right not held, the rook not on its corner, the destination occupied, or (queenside) the b-file square occupied	–
chess_castlingattacked	a castling-shaped king move, and the piece on <i>by</i> is an enemy attacking, in the prior, the king's square (<i>crossed</i> = 0) or the square it crosses (<i>crossed</i> = 1)	<i>crossed</i> , <i>by</i>
chess_kingattacked	in the new board, <i>king</i> holds side S 's king and the piece on <i>by</i> is an enemy that attacks it: the piece's geometry, and for a slider every square between is empty	<i>king</i> , <i>by</i>
chess_board	new[<i>sq</i>] is not what the move makes it: the moved or promoted piece on to; empty on from and on a pawn taken en passant; the rook moved by a castle; prior[<i>sq</i>] elsewhere	<i>sq</i>
chess_side	the new side to move is not $1 - S$	–
chess_castlingfield	the new castling rights are not the prior's with exactly the rights lost that the move touched (a king move loses both of that side's, a move from or to a corner loses that corner's)	–
chess_epfield	the new en passant square is not the square passed by a double push, or not 64 after any other move	–
chess_clock	the halfmove clock is not reset to 0 after a pawn move or capture, or not the prior plus one otherwise	–

The first two leaves are about the heads as records (the right slot, a decodable encoding);

`chess_malformed` uses only digit comparisons and never reads the board, so it stays live for a head whose fields would make the other leaves’ reads fail. The twelve `chess_*` kinds are the rules. Together they are designed to be *sound* (a legal move with its honest successor fires no leaf) and *complete* (any other well-formed tuple fires at least one, with some exhibit); in the code the first is a unit test over legal moves and the second is tested by fuzzing against an independent move generator. The exhibit is what makes each check bounded: king safety is a universal (“no enemy piece attacks the king”), and its falsification is one attacker, which the leaf verifies directly rather than searching.

The exhibits range over small sets: $j \in \{1, \dots, 6\}$, $crossed \in \{0, 1\}$, and squares in 0..63, range-checked in the leaf. A claimant who picks a wrong exhibit (a j pointing at an empty square, an attacker that does not attack) simply fails the leaf and can try another before the window closes; the challenge space is about 4,400 (kind, exhibit) pairs and the claimant searches it off-chain.

A.3. A worked disproof: a bishop through a pawn

This is scenario PC6 of the repository, run on regtest.

The moves. A (White) plays 1.e4, sealed at depth 1. At depth 2 B (Black) submits c8–e6: the bishop on c8 moves along the diagonal through d7, which holds Black’s own pawn. B signs the entry and publishes, as the new state, exactly what that move would produce if it were legal: the bishop on e6, c8 empty, side to move White, castling rights unchanged, en passant none, halfmove clock 1. The venue attests it (the venue checks signatures, not chess).

The dispute. A will not play on from an illegal position. Once the claim lock $T_2 + m$ has passed it broadcasts the thin absence claim `absent2` (189 vB), “B did not move at depth 2”. As a statement about the venue the claim is false, and B answers with `refute` (36,615 vB): the venue’s attestation of heads 1 and 2, the proposer fragments, and the Winternitz signature under K_2^{ref} that parks $\text{head}(1) \parallel \text{head}(2)$ in the output P_2 . The refutation is valid: the heads *were* attested. What it has done is put B’s move on chain in a form Script can read.

Finding the predicate. A runs the family off-chain on the parked tuple. Exactly one leaf fires: `chess_ray`, with $j = 1$. (`chess_board` does not: the new board is the honest consequence of the claimed move. Nor do `chess_mover`, since a bishop may move diagonally; `chess_destination`, since e6 is empty; or the field leaves, since B updated them correctly.)

The computation. In the register file, with the offsets above:

digits	value	meaning
81 (prior byte 32, low)	1	$S = \text{Black}$
$16 + 58 = 74$ (prior c8)	$11 (= 3 + 8)$	a black bishop
$16 + 51 = 67$ (prior d7)	$9 (= 1 + 8)$	a black pawn
184, 185 (new byte 36)	3, 10	$\text{from} = 16 \cdot 3 + 10 = 58$ (c8)
186, 187 (new byte 37)	2, 12	$\text{to} = 16 \cdot 2 + 12 = 44$ (e6)
188 (new byte 38, high)	0	no promotion

The preamble computes $r(58) = 7$, $f(58) = 58 - 56 = 2$ and $r(44) = 5$, $f(44) = 44 - 40 = 4$, so $\Delta f = 2$ and $\Delta r = -2$. The prior nibble at 58 is 11: non-zero, colour 1 = S , so $K = 11 - 8 = 3$, a bishop. The body of `chess_ray` is

```

guard := (df = 0 or dr = 0 or |df| = |dr|) and (df, dr) != (0, 0)
        and K != 2 and j >= 1 and j < max(|df|, |dr|)
if guard:
    step := sign(df) + 8 * sign(dr)           three doublings
    sq := from
    for i in 1..6: if i <= j: sq := sq + step   six gated additions
    result := prior[sq] != 0                  computed OP_PICK
else:
    result := 0

```

With the exhibit $j = 1$: the guard holds ($|2| = |-2|$, not a knight, $1 \leq 1 < 2$); $step = 1 + 8 \cdot (-1) = -7$; one gated addition gives $sq = 58 - 7 = 51$, d7; the read returns the prior nibble 9, which is non-zero. The body leaves 1; the cleanup drops everything else, and the spend succeeds with that 1 as the only stack element. Appendix A.4 shows the same computation opcode by opcode.

The spend. After the refutation has δ confirmations, A builds `disprove_chess_ray` spending P_2 : witness [1, the 195 reveal pairs from the refutation’s witness, σ_A], the leaf script and its control block. It confirms at 4,955 vB and pays the whole contract output to A. The three transactions total 41,759 vB. B’s checked `splitX` on the same output was locked for $\delta + \delta'$, so the output is gone before it could open.

Had A picked `chess_board` instead (a natural first guess for “the board is wrong”), the leaf would have failed whatever square it named; the regtest test `pos_chess_graph` checks that rejection before mining the ray disprove. The illegality is in the move, not in the bookkeeping, and the family has one leaf for each.

A.4. The same disproof, as bytes

The walk-through above is the program. What goes on chain is the transaction below, taken from the same regtest run; the raw transaction, the leaf’s full disassembly and every witness element are in the repository under `docs/paper/data/`. The transaction has one input (the refuted output P_2 , with `nSequence` = 6: $\delta = 6$ blocks on regtest), one output (820,000 sat to A), and a witness of 394 elements. It is 19,537 bytes, of which the witness is 19,441: weight 19,819, so 4,955 vB.

The leaf script: 14,791 bytes.

segment	bytes	
header: CSV and the claimant's key	37	
Winternitz steps, one per digit	13,650	195×70
Winternitz checksum finale	602	
the ray predicate (preamble and body)	391	
cleanup	111	
total	14,791	10,618 opcodes

The rule of chess is 391 bytes. Of the whole script, 96% is reading the parked digits back under the one-time key, which every leaf of the family repeats verbatim; only the predicate differs from leaf to leaf. In the listings, pushes are written $\langle n \rangle$ in decimal (the script uses OP_1...OP_16 or a minimal push) and the OP_ prefix is dropped after the first listing.

The header and the first Winternitz step, byte for byte (the step verifies the last checksum digit; the 194 others differ only in the 20-byte public-key hash):

OP_6 OP_CSV OP_DROP	$\delta = 6$
OP_PUSHBYTES_32 651920c198e3c13afe5c4995fd4e924a9c46bd5ebab08ab2a7df235c955b6b31	
OP_CHECKSIGVERIFY	σ_A
OP_SWAP OP_SIZE <20> OP_EQUALVERIFY	a 20-byte hash
OP_SWAP <15> OP_MIN OP_DUP OP_TOALTSTACK	the digit, for the checksum
<8> OP_2DUP OP_LESSTHAN	
OP_IF OP_DROP OP_TOALTSTACK OP_HASH160 $\times 8$	
OP_ELSE OP_SUB OP_TOALTSTACK OP_ENDIF	
OP_DUP OP_HASH160 $\times 7$	the chain to 15 hashes
OP_FROMALTSTACK OP_PICK	the hash at the digit's height
OP_PUSHBYTES_20 fa0d31c163b74e26212fc6057c974b14f8dcae03 OP_EQUALVERIFY	
OP_2DROP OP_2DROP OP_2DROP OP_2DROP	

The finale then recomputes $\sum_{i<192}(15 - d_i)$ from the altstack and compares it with the three checksum digits:

FROMALTSTACK DUP NEGATE (FROMALTSTACK TUCK SUB) $\times 191$ <2880> ADD
FROMALTSTACK (DUP ADD) $\times 4$ FROMALTSTACK ADD
(DUP ADD) $\times 4$ FROMALTSTACK ADD EQUALVERIFY

It leaves the 192 digits on the stack, digit 0 deepest and digit 191 on top, with the exhibit j below them all. Here is the predicate, all 381 opcodes, with the value each line leaves in this spend on the right.

<i>registers: fields gathered from the digits (a byte is high-16 + low)</i>		
<110> PICK	prior side	1
<109> PICK	prior castling	15
<109> PICK DUP ADD DUP ADD DUP ADD DUP ADD <109> PICK ADD	prior ep	20
<108> PICK DUP ADD DUP ADD DUP ADD DUP ADD <108> PICK ADD	prior halfmove	0
<0>	no move number	0
<19> PICK	new side	0
<18> PICK	new castling	15
<18> PICK DUP ADD DUP ADD DUP ADD DUP ADD <18> PICK ADD	new ep	64
<17> PICK DUP ADD DUP ADD DUP ADD DUP ADD <17> PICK ADD	new halfmove	1

<0>	no move number	0
<17> PICK DUP ADD DUP ADD DUP ADD DUP ADD <17> PICK ADD	from	58
<16> PICK DUP ADD DUP ADD DUP ADD DUP ADD <16> PICK ADD	to	44
<15> PICK	promotion	0
<i>preamble</i>		
<12> PICK	S	1
<1> OVER DUP ADD SUB	$1 - 2S$	-1
<4> PICK DUP <8> GREATERTHANOREQUAL	from; rank of from:	
OVER <16> GREATERTHANOREQUAL ADD OVER <24> GREATERTHANOREQUAL ADD		
OVER <32> GREATERTHANOREQUAL ADD OVER <40> GREATERTHANOREQUAL ADD		
OVER <48> GREATERTHANOREQUAL ADD OVER <56> GREATERTHANOREQUAL ADD		7
DUP DUP ADD DUP ADD DUP ADD ROT SWAP SUB	file = from - 8 rank	2
<5> PICK DUP <8> GREATERTHANOREQUAL	to; rank of to:	
OVER <16> GREATERTHANOREQUAL ADD OVER <24> GREATERTHANOREQUAL ADD		
OVER <32> GREATERTHANOREQUAL ADD OVER <40> GREATERTHANOREQUAL ADD		
OVER <48> GREATERTHANOREQUAL ADD OVER <56> GREATERTHANOREQUAL ADD		5
DUP DUP ADD DUP ADD DUP ADD ROT SWAP SUB	file of to	4
DUP <3> PICK SUB	Δf	2
<2> PICK <5> PICK SUB	Δr	-2
<10> PICK <196> SWAP SUB PICK	prior[from]	11
DUP ONOTEQUAL OVER <8> GREATERTHANOREQUAL <10> PICK NUMEQUAL	BOO LAND own piece?	1
OVER DUP <8> GREATERTHANOREQUAL DUP ADD DUP ADD DUP ADD SUB	its kind	3
OVER IF ELSE DROP <0> ENDIF	K (0 if not own)	3
<i>the ray body: the guard</i>		
<4> PICK <0> NUMEQUAL <4> PICK <0> NUMEQUAL BOOLOR	$\Delta f = 0 \vee \Delta r = 0$	0
<5> PICK ABS <5> PICK ABS NUMEQUAL BOOLOR	$\vee \Delta f = \Delta r $	1
<5> PICK ONOTEQUAL <5> PICK ONOTEQUAL BOOLOR BOO LAND	$\wedge$ not a null move	1
OVER <2> NUMNOTEQUAL BOO LAND	$\wedge K \neq 2$	1
<217> PICK <1> GREATERTHANOREQUAL BOO LAND	$\wedge j \geq 1$	1
<217> PICK <6> PICK ABS <6> PICK ABS MAX LESSTHAN BOO LAND	$\wedge j < \max(\Delta f , \Delta r)$	1
IF		
<4> PICK DUP <0> GREATERTHAN SWAP <0> LESSTHAN SUB	sign Δf	1
<4> PICK DUP <0> GREATERTHAN SWAP <0> LESSTHAN SUB	sign Δr	-1
DUP ADD DUP ADD DUP ADD ADD	step	-7
<14> PICK	$sq := \text{from}$	58
<1> <219> PICK LESSTHANOREQUAL IF OVER ADD ENDIF	$1 \leq j: sq += \text{step}$	51
<2> <219> PICK LESSTHANOREQUAL IF OVER ADD ENDIF	skipped	
<3> <219> PICK LESSTHANOREQUAL IF OVER ADD ENDIF	skipped	
<4> <219> PICK LESSTHANOREQUAL IF OVER ADD ENDIF	skipped	
<5> <219> PICK LESSTHANOREQUAL IF OVER ADD ENDIF	skipped	
<6> <219> PICK LESSTHANOREQUAL IF OVER ADD ENDIF	skipped	
<200> SWAP SUB PICK	prior[sq]	9
ONOTEQUAL SWAP DROP	occupied: the result	1
ELSE		
<0>		
ENDIF		

Each PICK depth is fixed when the leaf is built: 110 is digit 81 (the prior side) counted from the top of the 192; 217 and 219 reach past the file and the named values to the exhibit j ; 196 and 200 are the depths of the prior board's square 0 at those two reads, so that <base> SWAP SUB PICK fetches square s . The cleanup parks the result, drops the other 217 elements (the exhibit, the 192 digits and 24 named values) and brings the result back as the only stack element, which

is what a tapscript spend requires:

TOALTSTACK 2DROP ×108 DROP FROMALTSTACK

(`wrong_slot` and `chess_malformed` end instead as in the outline of §4.4, `OP_VERIFY`, the drops, `OP_1`; the two are equivalent.)

The witness. It has 394 elements; they are listed here from the bottom of the stack (element 0) up, with the element’s length in bytes.

0	1	01	the exhibit $j = 1$
1	20	690f6755ecad892ca2bc261c703a982368efb205	digit 0: hash
2	0	(empty)	digit 0: value 0
...			
135	20	3685302342eb8c77c2c76720d704e589830ec973	digit 67 (prior d7)
136	1	09	9: a black pawn
...			
149	20	2c31103666df2902a7453a27162bf6bb57197e08	digit 74 (prior c8)
150	1	0b	11: a black bishop
...			
369	20	d4b4a5068c7fd66efb32ec0a4c9ab8693e718840	digit 184
370	1	03	from, high nibble
371	20	51317f5d3588b80d2139d3b9564ffb10aaae956a	digit 185
372	1	0a	from = 0x3a = 58
373	20	150c592248ef4fdeb8e6a988f2e9e302e84d2ed7	digit 186
374	1	02	to, high nibble
375	20	ca10142fde07c2d6fb4e3fd0173ed0948bf5a46f	digit 187
376	1	0c	to = 0x2c = 44
...			
386	1	09	checksum digits (385–390):
388	1	01	
390	1	0b	0x91b = 2331
391	64	19ea82ef2e44ec8f...9d13474f	σ_A , Schnorr
392	14791	56b27520651920c1...	the leaf script
393	193	c050929b74c1a049...	control block: depth 5

Element $2i + 1$ is digit i ’s chain value and element $2i + 2$ its value as a minimal script number (so a 0 is the empty push). The 192 digit values, read in order, are the two parked heads; this is the *whole* of what B’s signed move contributes to the spend:

head(1) =	00010100	070c0000	42356324	11110111	00000000	00001000
	00000000	00000000	99999999	cabdebac	010f1400	0c1c0001
head(2) =	00010201	0b3a0000	42356324	11110111	00000000	00001000
	00000000	0000b000	99999999	ca0debac	000f4001	3a2c0002

Read it with the tables of §A: word0 (game 1, depth, mover), word1 (e2–e4 is $12 + 64 \cdot 28 = 0x070c$, c8–e6 is $58 + 64 \cdot 44 = 0xb3a$), then the board rank by rank from a1: 42356324 is White’s back rank, and in head(2) the eighth rank ca0debac has lost the bishop from c8 (the third nibble) while the sixth rank 0000b000 has gained it on e6. Rank seven, 99999999, still holds all eight black pawns, d7’s included. That pawn is the fact the whole spend proves.